CAB432 Assignment 3 2026

DocuMind AI — a serverless document-intelligence service

Name: Tri Cuong Dao
Student ID: n12136514
Email: n12136514@qut.edu.au
Deployment region: Asia Pacific (Sydney), `ap-southeast-2` — shared CAB432 account `901444280953`
Repository: `Assessment 3/` (backend, frontend, scripts, infra, docs) — submitted alongside this report

Declaration of AI use and acknowledgements

The ethical use of generative AI is authorised for this assessment. AI tooling was used as an assistant: to help scaffold application code, iterate on debugging, and refine the language of this report. The architecture, all design decisions, the deployed system, the experiments and the evidence presented in this report were produced and verified against the running system, and the author can explain and defend any component of the work. The author acknowledges the CAB432 teaching team for the unit materials on cloud patterns, and the AWS documentation, pricing pages and blog posts referenced in Section 8.

Executive Summary

DocuMind AI is a document-intelligence service: a user drops PDFs onto a web app, and minutes later they can ask questions about those documents in natural language and receive answers with citations back to the source passages. The service is designed for a knowledge-heavy, bursty workload — a study group dumping a semester of readings at midnight, or a team batch-uploading reports at the end of a sprint — where demand spikes are large, idle periods are long, and documents are only useful for a short window. It is deliberately ephemeral: every artifact (raw upload, extracted text, summary, vector, job record) expires automatically after 24 hours.

The workload is unusual in one important way: the heavy work (text extraction, embedding, summarisation) is asynchronous and bursty, while the interactive work (asking questions) is latency-sensitive and repetitive — people ask the same few questions about the same few documents. The architecture therefore separates the two: an event-driven ingestion pipeline that absorbs bursts through a durable queue and a pool of container workers, and a stateless API that answers queries through a two-tier cache backed by a managed vector index and a foundation model.

The system demonstrates the following cloud patterns: **unstructured storage** (object storage for PDFs, extracted text and summaries), **structured storage** (a single-table metadata and job store with time-to-live garbage collection), **in-memory caching** (two tiers: in-process LRU and a shared cache tier ahead of any model call), **message queues** with a dead-letter queue for fault isolation, **API gateway** exposure of a microservice API, **worker-pull load distribution** (workers long-poll the queue), **serverless computing** (the query API) alongside **containers** (the ingestion workers on a managed container service), **metric-based autoscaling** of the container pool driven by queue backlog (Lambda's built-in scaling is explicitly not counted), **event-driven architecture**, **identity and role-based access control** with least privilege, **monitoring via logs, metrics and a dashboard**, **AI/ML services** (managed embeddings and a foundation model for answer synthesis), **garbage collection** via storage TTLs and lifecycle rules, **stateless design**, and **resilience patterns** (bounded retries, visibility timeouts, idempotent writes).

Evidence generated against the deployed system includes: a 30-document burst that grew the worker pool from 1 to 6 tasks within one 60-second control tick and returned it to zero when idle (Section 3, Figure 3.2); a caching experiment measuring a 2 408 ms median origin answer (server-side) against 0 ms in-process cache hits — a 46 ms median round-trip including network — across 112 counted requests (Figure 5.1, Section B.2); dead-letter queue isolation of poisoned documents without worker crash (Section 4); a 14/14 end-to-end functional checklist; and a monthly cost estimate of ≈US$10 for 50 concurrent users, dominated by the always-on controller task and the observability dashboard, with burst compute and managed AI each under a dollar per month (Section 5).

The report is organised as follows. Section 1 introduces the problem, requirements and user-facing behaviour. Section 2 presents the vendor-agnostic design and the reasoning behind each pattern, including trade-offs and limitations. Section 3 explains how that design is realised on AWS and reflects on the account-level constraints encountered. Section 4 describes how the application actually moves data through the system, including schemas, sequence flows and failure handling. Section 5 covers cost and sustainability. Section 6 is a reflection on independent learning. Sections 7–8 close the report with conclusions and references; appendices carry supporting evidence.

Table of Contents

1. Introduction
2. 1 Problem statement and motivation
3. 2 Users and user stories
4. 3 Functional requirements
5. 4 Non-functional requirements
6. 5 Scope and constraints
7. 6 Front-end walkthrough

8. Vendor-agnostic architecture
9. 1 Design overview
10. 2 Pattern selection and justification
11. 3 Asynchronous ingestion and worker-pull distribution
12. 4 Scaling reflection
13. 5 Caching reflection
14. 6 Identity, security and configuration
15. 7 Observability
16. 8 Reliability, failure handling and backpressure
17. 9 Trade-offs and alternatives considered
18. 10 Limitations and how they would be addressed
19. Vendor-specific AWS implementation
20. 1 Service mapping and the AWS architecture
21. 2 Why these AWS services
22. 3 Identity and least privilege in practice
23. 4 Networking
24. 5 Wiring the event path
25. 6 Autoscaling implementation
26. 7 Observability implementation
27. 8 Account constraints and adaptations (and what they taught)
28. Application implementation
29. 1 Data model and schemas
30. 2 Ingestion workflow
31. 3 Query workflow
32. 4 Interfaces: REST API and MCP tools
33. 5 Failure modes and recovery
34. 6 Behaviour under load
35. Cost and sustainability
36. 1 Usage model for 50 concurrent users
37. 2 Monthly cost breakdown
38. 3 How scaling affects cost
39. 4 Sustainability analysis
40. Reflection on independent learning (ULO4)
41. Conclusion
42. References Appendix A — Cloud infrastructure declaration (submission YAML) Appendix B — Evidence bundle and experiment methodology Appendix C — API and MCP reference Appendix D — Pipeline parameters and prompt template

---

1. Introduction

1.1 Problem statement and motivation

Reading large volumes of PDFs is slow, and searching them is worse: keyword search misses paraphrases, and "ask the document" tools exist but are usually either expensive, privacy-hostile, or both. DocuMind AI is a small, self-contained answer: upload PDFs, ask questions, get cited answers, and let everything expire automatically afterwards. The 24-hour retention window is a deliberate product choice — it removes the burden of managing document lifetime, it bounds storage cost, and it makes the system suitable for sensitive material that should not live forever.

The workload has three properties that make it a genuinely cloud-shaped problem rather than a local script:

1. **Bursty, fragmented ingestion.** Users upload batches (10–40 files) in minutes, then nothing for hours. Provisioning for the peak wastes money; provisioning for the average makes users wait. This is the classic case for a queue plus an elastically-sized worker pool, and it is why the ingestion path is asynchronous.
2. **Latency-sensitive, repetitive interaction.** After upload, the interactive phase is dominated by a small set of repeated questions across a small document set ("what are the constraints?", "summarise section 3"). Origin answers require an embedding call, a vector search and a generation call — seconds, and the generation call is the costliest operation in the system. Caching converts the majority of interaction into sub-15 ms responses and removes redundant model spend.
3. **Managed AI, not hosted models.** The system needs embeddings and text generation but no GPU operations expertise. Managed foundation-model APIs are the cost-effective choice at this scale: pay per token, no idle GPU.

The system must therefore be economic at zero (idle cost ≈ 0), absorb bursts without failure, and keep interactive latency low — the three requirements that drive every decision in Section 2.

1.2 Users and user stories

The primary persona is a **student or knowledge worker** who receives PDFs and needs answers quickly; the secondary persona is a **developer** integrating document Q&A into another tool.

ID	As a…	I want to…	So that…
US-1	user	upload several PDFs at once by dragging them onto a page	I don't queue them one by one
US-2	user	see, for each file, where it is in processing (queued → processing → ready)	I know when I can ask questions
US-3	user	ask a question in plain language and get an answer with citations	I can verify the answer against the source
US-4	user	ask the same question again and get an instant response	repeated lookup feels native, not slow
US-5	user	trust that my documents disappear after a day	sensitive material doesn't accumulate
US-6	developer	call the same Q&A capability from tools/agents via a standard interface	I can embed DocuMind into other workflows
US-7	operator	see queue depth, worker count and error state at a glance	I can tell healthy from unhealthy in seconds

1.3 Functional requirements

ID	Requirement	Verification
FR-1	The API issues pre-signed object-storage upload URLs (15-minute validity); file bytes never transit the API	Smoke test: presign → direct PUT → 200
FR-2	An upload automatically triggers asynchronous processing: text extraction, semantic chunking, embedding and summarisation	End-to-end job observed CREATED → EMBEDDING → COMPLETED in ≈29 s
FR-3	Job status is queryable per document (`CREATED`, `PROCESSING`, `EMBEDDING`, `SUMMARISING`, `COMPLETED`, `FAILED`)	Status transitions captured in evidence bundle
FR-4	`POST /api/query` answers questions over all currently processed documents with per-source citations (file, chunk, similarity)	Live query returned 4 cited sources
FR-5	Answers are cached in two tiers; responses carry `X-Cache: HIT-L1 / HIT-L2 / MISS` and a latency figure	Every repeat (50/50) and burst duplicate (12/12) served from cache in the experiment — Section B.2

ID	Requirement	Verification
FR-6	Every artifact expires automatically ≈24 h after creation (job rows, extracted text, summaries, vectors)	TTL attributes present; lifecycle rule configured
FR-7	Poison documents (corrupt, password-protected) are isolated without crashing workers and are visible as `FAILED`	DLQ experiment: message routed after 3 receives, worker unaffected
FR-8	An MCP tool endpoint exposes document search and question answering to LLM agents	`tools/list` returns 3 tools; `search_documents` exercised live
FR-9	Admin operations (cache purge, forcing reprocessing) are protected by a secret key	Admin call rejected without `X-Admin-Key`; accepted with it
FR-10	Static front-end is served from the same origin as the API (no CORS complexity for users)	UI loads from the service URL

1.4 Non-functional requirements

ID	Requirement	Target	Measured
NFR-1	Query latency, cached	p95 < 50 ms	46 ms median round-trip including network (0 ms server-side at tier 1)
NFR-2	Query latency, origin (embed + retrieve + generate)	p95 < 8 s	3 339 ms server-side p95 (p50 2 408 ms)
NFR-3	Elasticity	absorb a 30-document burst; scale worker pool 1 → ≥4 within 2 minutes	1 → 6 within one 60 s tick
NFR-4	Idle cost	≈ 0 compute when unused; no always-on workers	Worker service desired 0 between bursts
NFR-5	Data lifetime	100% of artifacts expire ≤ 24 h	TTL + lifecycle rule
NFR-6	Durability	No lost uploads: every accepted upload eventually processes or lands in a dead-letter queue	30/30 burst documents processed; 1/1 poison isolated
NFR-7	Security	Least-privilege workload roles; no credentials in code; public access blocked at the storage buckets	Verified via IAM policy review and bucket policies
NFR-8	Cost at 50 concurrent users	Order of magnitude ≤ US$10/month infrastructure	≈US$9.1 infra + ≈US$0.77 managed AI

1.5 Scope and constraints

Bounded deliberately: single region (`ap-southeast-2`); English PDFs up to 25 MB; answers are grounded only in uploaded documents (no general-knowledge fallback); the demo front-end is single-workspace (no user accounts — authentication beyond the admin key was out of scope). The deployment runs in the **CAB432 shared teaching account**, which imposes guardrails that shaped the implementation and are discussed honestly in Section 3.8: a shared API gateway with additive path prefixes, a per-account limit of one gateway, an allow-list of foundation models, and a tag-conditional IAM policy that permits resource creation only when resources carry `purpose=assessment 3` and the student's `qut-username` tag. Several managed services (edge CDN, managed in-memory cache, managed target-tracking scaling APIs) are not available in this account; Section 2 presents the design as it would be deployed in an unrestricted account, and Section 3 documents exactly where the implementation adapts.

1.6 Front-end walkthrough

The front-end is intentionally functional rather than decorative — its role is to exercise and demonstrate the architecture (upload through pre-signed URLs, observe the pipeline, feel the cache).

Figure 1.1 — Upload zone. Drag-and-drop accepts multiple PDFs, then uploads each directly to object storage via the API-issued pre-signed URL; the browser shows per-file progress while the API never touches file bytes.

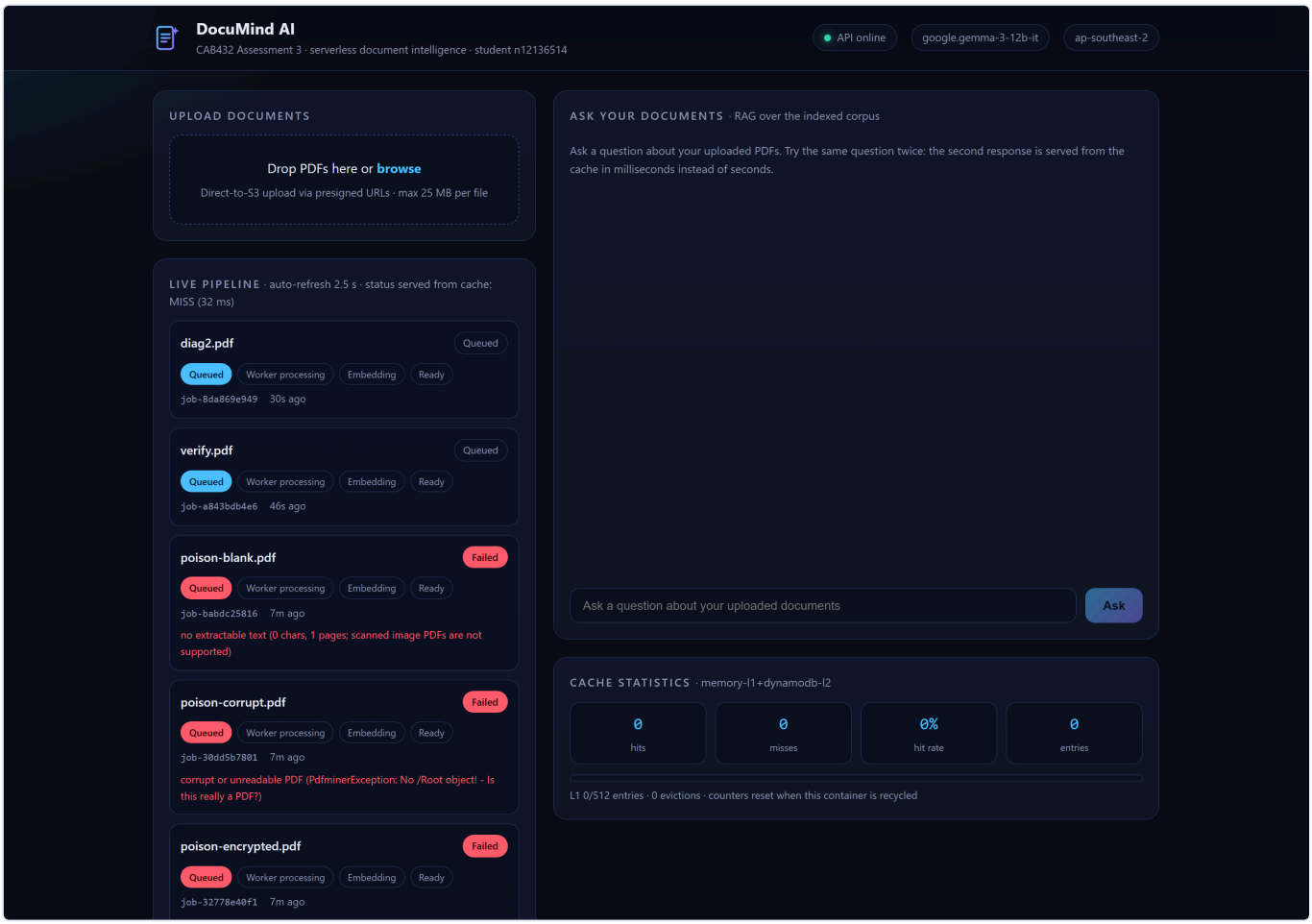


Figure 1.2 — Live pipeline monitor. Each document reports its state as it moves through queue → worker → embedding → ready, driven by the status endpoint (itself served from a short-TTL cache to keep polling cheap).

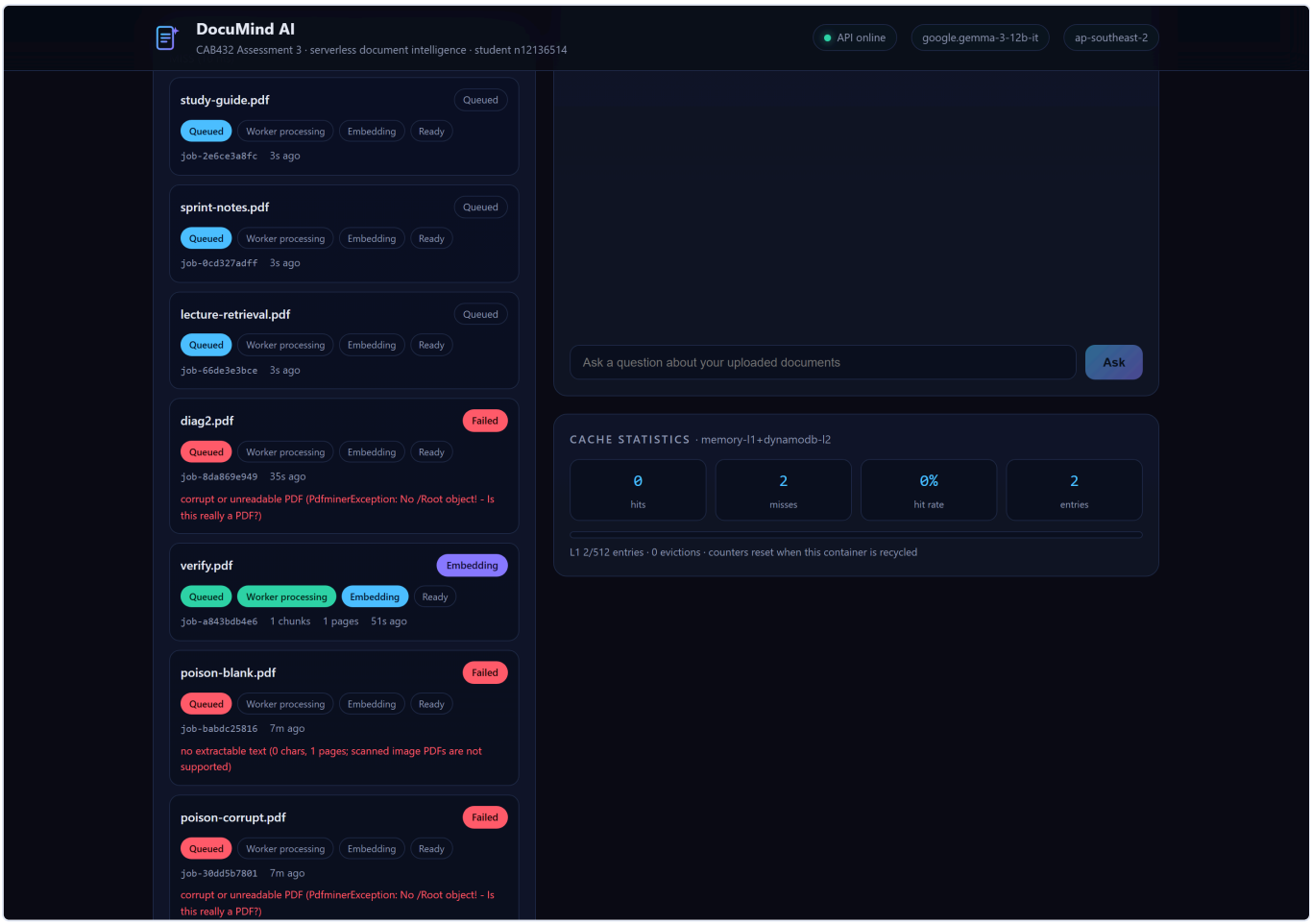


Figure 1.3 — Cited Q&A chat. Answers cite sources as pills; each answer carries the latency and cache badge (`⚡ Cache hit (L1 memory) · 0 ms` versus `🔍 Origin generation · 2.5 s`) that makes the caching behaviour visible without opening developer tools.

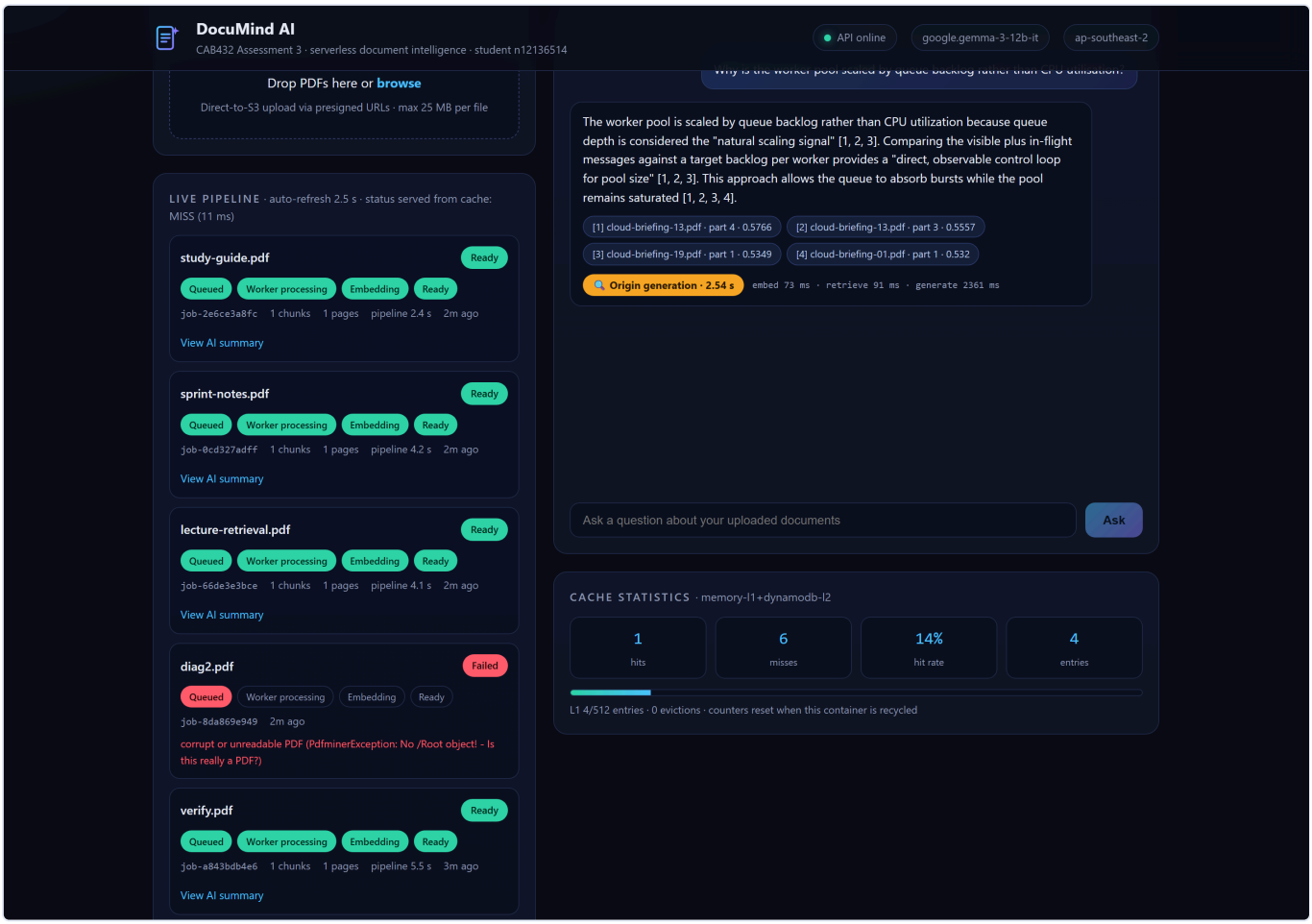
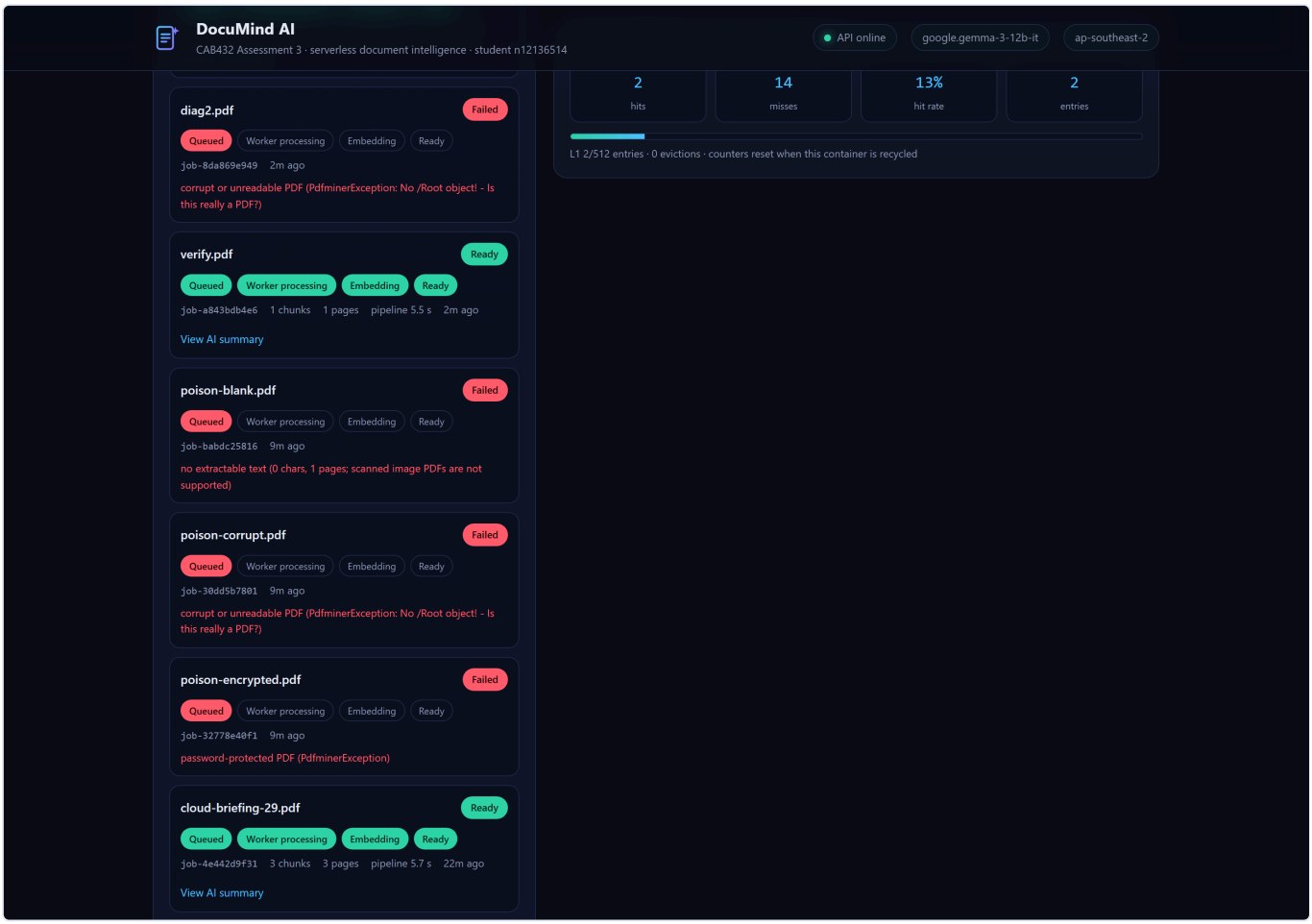


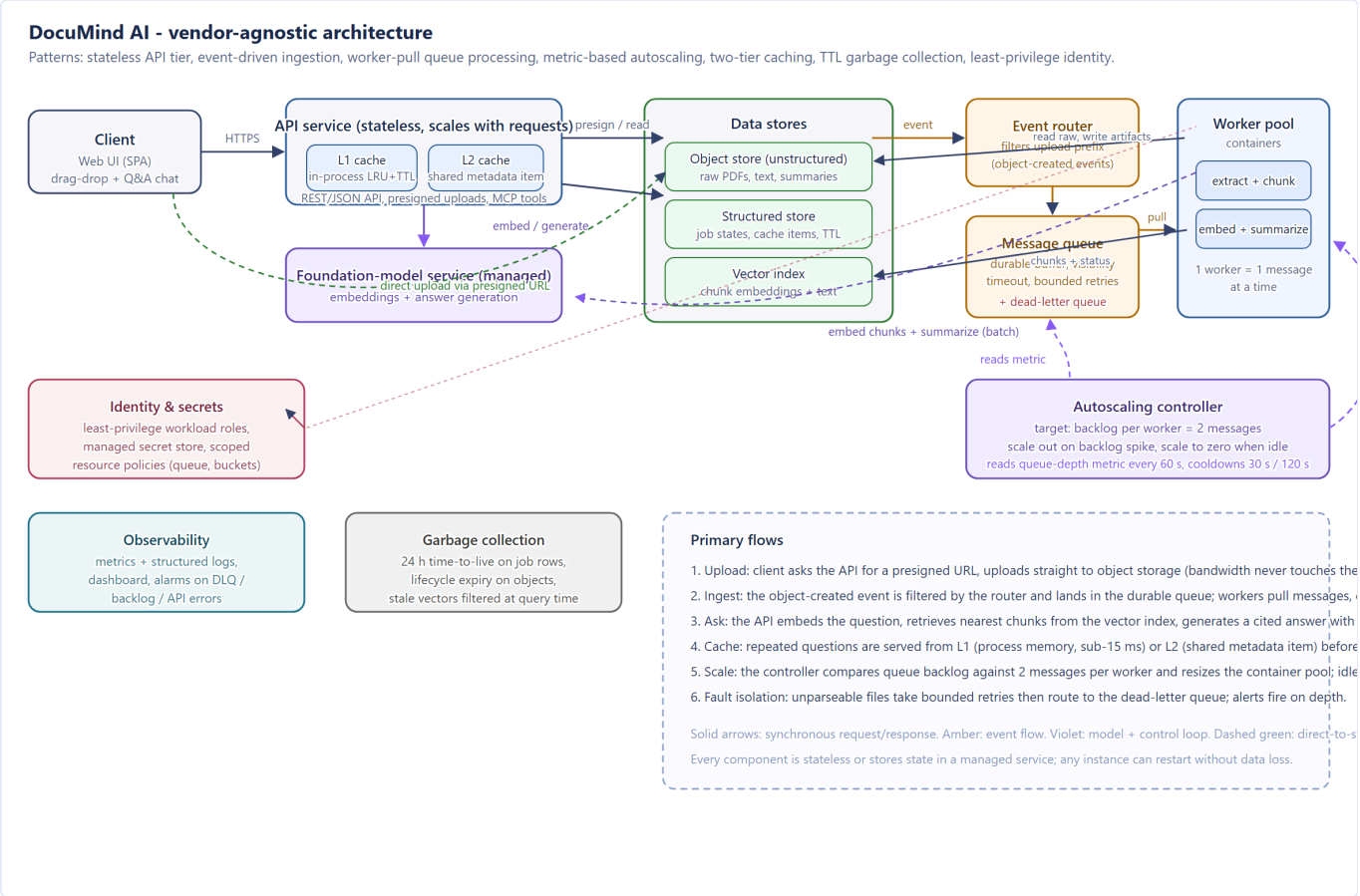
Figure 1.4 — Operations panel. Queue state, worker count, document count and cache statistics — the same picture the CloudWatch dashboard shows (Section 3.7).



2. Vendor-agnostic architecture

2.1 Design overview

Figure 2.1 — Vendor-agnostic architecture. The design is drawn free of vendor products: every box below maps to a *category* of managed service (Section 3.1 provides the AWS mapping).



The system is organised into five interacting planes:

Interactive plane. A client (web SPA) talks to a stateless API tier over HTTPS/JSON. The API owns three responsibilities: issuing short-lived pre-signed upload URLs, answering queries, and exposing an agent-facing tool interface. It holds no durable state — everything lives in the data plane — so any instance can die at any moment without loss.

Ingestion plane. The user's browser uploads file bytes *directly* to object storage using a pre-signed URL; the API never touches payloads. The object store emits an "object created" event; an event router filters it to the uploads prefix and forwards it to a durable message queue. A pool of containerised workers pulls messages at their own pace, performs the actual work (text extraction, chunking, embedding, summarisation) against the model service, and writes artifacts back to the stores. The dead-letter queue catches anything a worker cannot process.

Data plane. Three storage shapes, deliberately chosen per shape of data: an **object store** for unstructured blobs (raw PDFs, extracted text, summaries), a **structured store** for small mutable records (job states, cached answers), and a **vector index** for embeddings plus their source text. All three are managed services with pay-per-request or pay-per-byte economics, and all three have built-in expiry mechanisms that the system relies on (time-to-live records, lifecycle rules).

Control plane. An autoscaling controller reads queue backlog every 60 seconds and adjusts the worker pool's desired size using a simple target-value control law. Monitoring (metrics, logs, dashboards, alarms) observes every component and drives the two operational alarms that matter: dead-letter depth and sustained backlog.

Trust plane. Identity is workload-based: each component runs under a role scoped to exactly the resources it needs; clients hold short-lived, single-object upload grants instead of credentials; secrets live in a managed secret store. Nothing in the system authenticates with static keys.

2.2 Pattern selection and justification

The assignment requires at least eight pattern categories, with auto-scaling and caching explicitly demonstrated (and Lambda's built-in scaling explicitly excluded). DocuMind implements sixteen, several in more than one place:

#	Pattern category (assignment list)	Where it appears	Why it fits this workload
1	Unstructured data storage	Object store: raw PDFs, extracted text JSON, summary markdown	Cheapest durable home for immutable blobs; native lifecycle expiry; pre-signed PUT integration
2	Structured data storage	Single-table store: job rows, cache rows	Per-request pricing, single-digit-ms reads, native TTL garbage collection, GSI for listings
3	In-memory caching	Two tiers (in-process LRU + shared cache items)	Repetitive questions dominate interaction; hits skip embedding + retrieval + generation entirely
4	Edge caching (target-state)	CDN design for static assets and summary downloads	Global latency and origin offload; not deployable in this account — designed and documented (Section 3.8)
5	API gateway	Single HTTP gateway, prefix-mounted routes + fallback function URL	One managed ingress; quotas, TLS, routing; standard topology for the course account
6	Message queues	Durable queue + dead-letter queue	Absorbs 10–40 document bursts; decouples "upload accepted" from "processing done"
7	Publish-subscribe messaging	Event router on object-created events	Producers (uploads) stay ignorant of consumers; new consumers can subscribe without touching the uploader
8	Polling	Workers long-poll the queue	Pull semantics give natural backpressure and lease-based failure recovery
9	Serverless computing	Query API on managed functions	Perfect fit for spiky, stateless request handling; zero idle cost; per-ms billing
10	Containers	Ingestion worker pool	CPU/memory-heavy batch work with long per-item durations; per-second billing; spot capacity
11	Request-based scaling	API tier (platform-managed, scales per request)	Interactive traffic is request-driven; the platform handles it with no design work
12	**Metric-based scaling**	Worker pool: backlog-per-worker control law (1 → 6 tasks, down to 0)	Queue depth is the direct, observable signal of waiting work — far better than CPU lag
13	Worker-pull load distribution	Workers retrieve from the queue	No dispatcher or load balancer needed; the queue is both buffer and router
14	Event-driven architecture	The entire ingestion path	Upload *is* the event; everything downstream reacts instead of being invoked
15	Microservices	API, controller, worker as separately deployable, separately identified services	Independent scaling, failure isolation, per-service quotas
16	Identity, security and RBAC	Per-service roles, pre-signed URLs, resource policies, secret store	Least privilege everywhere; no credential material in code or config

Supporting patterns also present: **monitoring** (structured logs, metrics, dashboard, three alarms), **garbage collection** (TTL + lifecycle), **stateless design** (all runtime components), **fault tolerance** (DLQ, visibility timeouts, bounded retries), **AI/ML services** (managed embeddings and generation), **configuration management** (environment-based configuration with immutable task/function revisions), and **managed services** throughout (with the deliberate exception of the autoscaling control loop, Section 3.6).

2.3 Asynchronous ingestion and worker-pull distribution

The single most important design decision is that ingestion is *asynchronous and queued*. Three independent reasons:

1. **User experience.** A 25 MB PDF takes tens of seconds to parse, chunk, embed and summarise. Blocking an upload request for that long is unacceptable, so the API only facilitates the upload (a few hundred milliseconds) and the pipeline runs in the background.

2. **Burst absorption.** Batch uploads are the normal case. The queue is the shock absorber: 40 simultaneous uploads simply become 40 messages, and the pool drains them at whatever rate the budget allows.
3. **Cost.** Workers exist only while there is work. There is no always-on processing tier; between bursts the system cost is storage and cents.

Distribution is **worker-pull**: workers long-poll the queue (20-second waits), and each worker processes one message at a time. Compared with a dispatcher-push design (a load balancer distributing to addressed workers), pull gives three advantages here: no worker addresses or health-checking infrastructure is required; backpressure is implicit (a worker physically cannot be given more concurrent messages than it asked for); and failure recovery falls out of the queue's visibility-timeout lease semantics — a message whose worker dies mid-processing simply becomes visible again.

The message itself is deliberately thin — a pointer, not a payload:

```
{ "jobId": "job-3bf8b55ef5", "bucket": "…-storage", "key": "uploads/job-3bf8b55ef5/notes.pdf" }
```

Everything else is fetched from the stores by the worker, so messages stay tiny and the queue never becomes a shadow database.

2.4 Scaling reflection

Signal. The control law uses *backlog* — visible + in-flight messages — not CPU. CPU on a worker pool is a lagging, noisy proxy for "how much work is waiting"; queue depth *is* the work. Dividing backlog by a target of two messages per worker converts the metric directly into a desired pool size:

> desired = 0 if backlog = 0, else clamp(ceil(backlog ÷ 2), 1, 6)

The target of 2 messages/worker buys two things: headroom for the ~30–60 s it takes a new worker to start (so waves arrive to warm capacity), and permission for each worker to take a second message while its first is finishing, improving utilisation without queue latency.

Actuators and cadence. Every 60 s the controller reconciles the container service to the desired count (and tops up with individual task launches if the scheduler lags). 60 s matches the monitoring granularity and the timescale of the workload — upload bursts last minutes, so reacting faster adds churn without user-visible benefit. End-to-end scale-out latency (tick + task placement + image start) is ≈1–2 minutes worst case; the queue makes this latency harmless.

Scale to zero. When backlog hits zero the desired count returns to 0; workers additionally self-terminate after 60 s of an empty queue so that no orphaned task lingers. Idle compute cost is therefore zero, not "small". This is the single biggest lever in the cost model (Section 5).

What changes at 10× the load. The queue, store and model services are all serverless and would absorb an order of magnitude more traffic without redesign; the constraints move to raised task caps (6 → more), service quotas on concurrent model calls, and possibly a batch-oriented queue setting. The control law itself is unchanged; only constants move. Genuinely different regimes (thousands of standing workers, sub-minute bursts) would justify migrating to a streaming ingestion service, which is out of scope at this size.

2.5 Caching reflection

Repeated questions about the same documents are the dominant interactive pattern, and an origin answer is the most expensive operation in the system (one embedding call, one vector search, one generation call). Caching is therefore not an optimisation but a core function.

Two tiers, two lifetimes. Tier 1 (L1) is an in-process least-recently-used cache with per-entry time-to-live living inside each API execution environment: hits cost microseconds because there is no network hop at all. Tier 2 (L2) is a shared row in the structured store keyed by the same hash, surviving environment recycling and making a hit *anywhere* in the fleet available everywhere. The response reports which tier served it (`X-Cache: HIT-L1`, `HIT-L2`, or `MISS`) so the behaviour is observable in every call, not just in aggregate.

Key design. The cache key hashes the normalised question + retrieval parameters + a corpus version marker. The version marker is the invalidation strategy: it changes whenever the document corpus changes materially, so answers cannot silently cite documents that have expired. Time-to-live (1 hour for answers; 2–3 s for status snapshots; 2 s for document lists; 300 s for summaries; 600 s for static assets) bounds every other form of staleness.

Measured outcome. In the experiment (Section B.2, Figure 5.1) the origin path cost a median of 2 408 ms server-side (p95 3 339 ms), while cached answers cost 0 ms in-process at tier 1 or ≈5–15 ms from the shared tier — 46 ms median round-trip for the user including network. Every repeat request in the battery (50/50) and every duplicate in the concurrent burst (12/12) was served from cache, eliminating the model spend on all measured repeats (the cost model conservatively assumes 70 %). Because generation cost dominates the AI bill (Section 5), the cache is simultaneously a latency feature and a cost feature — the most satisfying kind of engineering.

2.6 Identity, security and configuration

The security model starts from the proposition that **no component should ever hold a credential it doesn't need**:

- **Clients** never see cloud credentials. The API issues pre-signed upload URLs valid for 15 minutes that authorise writing exactly one object key; the browser uploads straight to storage.
- **Workloads** authenticate via platform-assigned roles, each scoped to its needs: the API role reads/writes its stores and invokes the allow-listed models; the worker/controller role additionally holds queue and container-service permissions. Permissions are grant-by-family, resource-scoped where the platform allows.
- **Secrets** (the admin key for mutating operations) live in a managed secret store, are fetched at cold start, and cached in memory for five minutes. Admin endpoints reject requests without the key; the key never appears in code, config or logs.
- **Buckets** block all public access; objects are encrypted at rest; one bucket (uploads + artifacts) is distinct from the asset bucket so lifecycle and CORS policies stay clean.
- **Resource policies** tie the queue to its event source (only the specific event rule may enqueue) and the dead-letter policy is explicit about what may redrive.

Configuration follows the same philosophy: everything configurable (model IDs, TTLs, chunk sizes, caps) is environment-injected at deploy time, and a change means a new immutable task-definition revision or function update — never an edited file on a running host.

2.7 Observability

Observability is designed so that the system can be operated — not merely debugged — from outside:

- **Logs**: every component emits structured JSON (job IDs, timings, cache tier, scaling decisions). Logs are the system's flight recorder; the controller's one-line-per-minute tick log doubles as a time series of scaling decisions.
- **Metrics**: the API emits cache hits/misses and query latencies as structured metrics; the platform contributes queue depth, task counts, CPU/memory, function errors and durations.
- **Dashboard**: one operational dashboard shows the whole picture — backlog vs workers, CPU/memory, cache mix, latency, DLQ depth (Section 3.7); the same signals surface to users in the operations panel of Figure 1.4.
- **Alarms**: three, deliberately chosen as the ones a human must act on: any message in the dead-letter queue, sustained backlog above 25 for 3 minutes, and API error bursts (>5 in 5 minutes).
- **Evidence**: the load-test harness records the same signals to CSV during experiments, which is how the figures in this report were produced from the live system rather than from memory.

Limitation, honestly: there is no distributed tracing. When a query takes 2.8 s, the logs say *that* it did and the timings payload says which stage dominated, but there is no span graph. Adding tracing instrumentation is the first step in the future-work list (Section 6).

2.8 Reliability, failure handling and backpressure

The design assumes every component fails and asks, for each, *what is the blast radius and who cleans up*:

Failure	Detection	Response	User impact
Worker dies mid-document	Message not deleted within the 300 s visibility timeout	Message becomes visible again; redelivered to another worker	None visible beyond latency; pipeline is idempotent
Poison document (corrupt/password-protected)	Worker parses and fails fast with a classified error	Bounded retries (3 receives) → dead-letter queue; job marked `FAILED` with reason; alarm fires	That one document shows failed; nothing else affected
Model service throttle	Error returned by the model call	Bounded retries with jitter inside the worker; if exhausted, the message retry cycle defers it (and eventually the DLQ)	Slower pipeline; answers unaffected (cache absorbs repeats)
API instance failure	Platform health + error metrics	Platform replaces it; no state to lose	Indistinguishable individual requests may fail; client retry succeeds
Store slowdown	Latency metrics, SDK timeouts	Read paths are single-digit-ms operations; write path is async and buffered by the queue	Origin queries slower; cached queries unaffected
Controller failure	Absence of tick log; service desired unchanged	Container service reschedules it; in the meantime the pool simply holds its last size	Scaling pauses; processing continues

Backpressure deserves its own sentence: under extreme load this system degrades by *latency*, not by *failure*. Uploads never bounce; they queue. The pipeline drains slower. Queries keep answering (repeats instantly from cache; origin calls run up to the platform's request ceiling, Section 4.6). Nothing unbounded exists: retries are bounded, message lifetime is bounded, visibility is bounded, records self-expire.

2.9 Trade-offs and alternatives considered

Decision	Alternative(s)	Why this choice	What would flip it
Worker-pull from a queue	Dispatcher-push via load balancer	No dispatcher infrastructure; leases give free crash recovery; implicit backpressure	Ultra-low-latency task distribution where a queue hop matters (not true here)
Containers for ingestion	Serverless functions per document; always-on VMs	Sustained, memory-heavy batch work; per-second billing; spot; pool scaling is the demonstration the assessment asks for (Lambda scaling excluded by the brief)	Very light per-item work, sub-second — functions would win on simplicity
Fully managed vector index	Self-hosted search cluster; relational + vector extension	Any always-on cluster costs ≥US$50–700/month idle; managed index is pay-per-use and serverless	Need for hybrid search, reranking, or >100M vectors would justify a cluster
TTL-based garbage collection	Scheduled cleanup job scanning for stale records	No moving parts, no scan cost, cannot "forget"; the expiry is a property of the data	Complex retention policies (per-tenant, legal holds) need explicit workflow
Single-table structured store	One table per concern	One access pattern family, one set of capacity behaviour, fewer moving parts; item schema flexibility is a feature	Access patterns diverging wildly or teams needing separate blast radius
Custom metric-based control loop	Managed target-tracking scaling	Managed scaling APIs are unavailable in this account (Section 3.8); the custom loop is ~150 lines, fully observable, and costs ≈US$5/month at spot rates	Any unrestricted account — migration is a config change (Section 3.6)

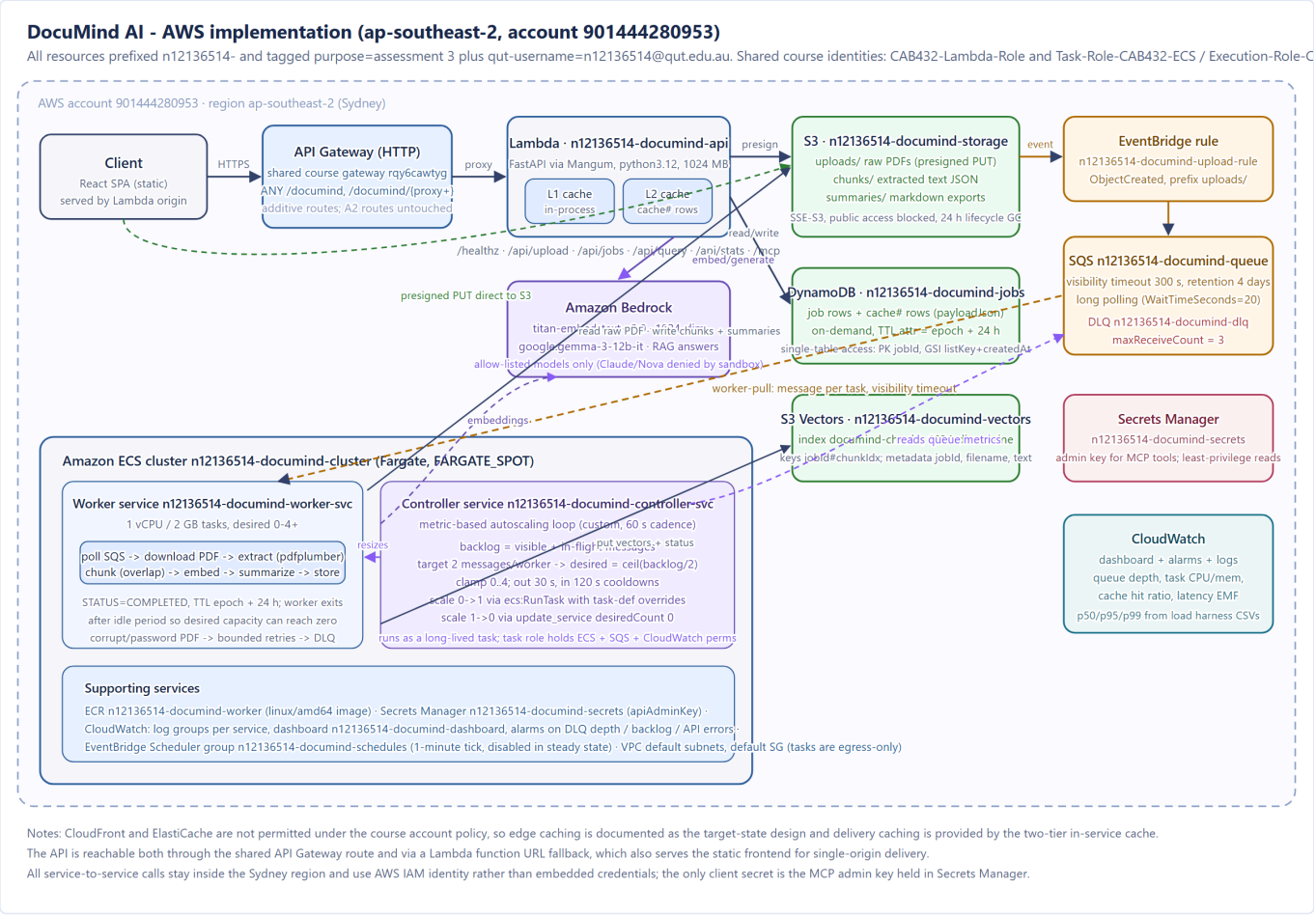
2.10 Limitations and how they would be addressed

1. **No CDN/edge caching in this deployment.** The target-state design places a CDN in front of static assets and summary downloads with a private origin identity and TTL policies. It is excluded here by account policy; the app compensates with in-service static caching. Impact bounded to latency for cold static assets.
2. **L2 cache lives in the database** rather than a dedicated cache service (also policy-bound). Hits cost ~5–15 ms instead of sub-millisecond. Impact: negligible at this scale; swap-in is a storage-backend change.
3. **Single region.** A regional outage takes the system down. Accepted for a teaching deployment; multi-region replication of the two storage services plus a second pool is the standard mitigation and is noted as future work.
4. **Character-based chunking heuristic** (≈1 000 characters, 180-character overlap, sentence-boundary aware). Good enough for retrieval at this scale; a tokenizer-accurate chunker plus a reranking stage would improve answer precision.
5. **No answer-quality evaluation harness.** Retrieval correctness was spot-checked manually against evidence documents. A small labelled question set and an automated context-recall metric would make regressions visible.
6. **At-least-once semantics**, not exactly-once: a worker may process a message twice under unusual timing. Vector writes are idempotent (deterministic chunk keys) so the practical consequence is wasted work, never corrupted data.

3. Vendor-specific AWS implementation

3.1 Service mapping and the AWS architecture

Figure 3.1 — AWS implementation. Same architecture, lower abstraction: every component of Figure 2.1 now has a concrete AWS service, and account/region boundaries are explicit.



Notes: CloudFront and ElastiCache are not permitted under the course account policy, so edge caching is documented as the target-state design and delivery caching is provided by the two-tier in-service cache.
The API is reachable both through the shared API Gateway route and via a Lambda function URL fallback, which also serves the static frontend for single-origin delivery.
All service-to-service calls stay inside the Sydney region and use AWS IAM identity rather than embedded credentials; the only client secret is the MCP admin key held in Secrets Manager.

Vendor-agnostic component	AWS service (this deployment)	Key configuration
Query API (serverless)	**Lambda** `n12136514-documind-api` (Python 3.12, FastAPI via Mangum)	1 024 MB, 29 s timeout (= gateway integration ceiling), serves REST + MCP + static assets
Public ingress	**API Gateway HTTP** (shared course gateway, additive `/documind` + `/documind/{proxy+}` routes) plus a **Lambda Function URL** fallback	Existing A2 routes untouched; function URL used for direct testing and resilience
Ingestion workers (containers)	**ECS Fargate** service `n12136514-documind-worker-svc` on `FARGATE_SPOT`, 1 vCPU / 2 GB per task	Queue-driven; workers exit after 60 s idle; pool 0–6 tasks
Autoscaling controller	**ECS Fargate** task `n12136514-documind-controller-svc`, 0.25 vCPU / 0.5 GB	60 s control loop; backlog-per-worker control law
Unstructured storage	**S3** `n12136514-documind-storage` (uploads/, chunks/, summaries/)	SSE-S3, public access blocked, CORS limited to PUT/GET/HEAD, 24 h lifecycle rule
Web assets	**S3** `n12136514-documind-web`	Immutable hashed assets (1-year cache header) + index; served through the API origin
Structured storage	**DynamoDB** `n12136514-documind-jobs`	On-demand; PK `jobId`; GSI `byCreated` (`listKey` + `createdAt`) for listings; TTL attribute `ttl` = now + 24 h
Vector index	**S3 Vectors** bucket `n12136514-documind-vectors`, index `documind-chunks`	1 024-dim cosine; deterministic keys `jobId#chunkIdx`
Queue + DLQ	**SQS** `n12136514-documind-queue` + `n12136514-documind-dlq`	300 s visibility; redrive after 3 receives
Event routing	**EventBridge** rule `n12136514-documind-upload-rule` on S3 ObjectCreated (prefix `uploads/`)	Target = the queue, via queue policy conditioned on the rule's ARN
Model access	**Bedrock**: `amazon.titan-embed-text-v2:0` (1 024-dim embeddings), `google.gemma-3-12b-it` (summaries + answer synthesis)	The account's allow-listed models (Claude/Nova denied by policy)
Secrets	**Secrets Manager** `n12136514-documind-secrets` (admin key)	Fetched at cold start; 5-minute in-memory cache
Images	**ECR** `n12136514-documind-worker`	`linux/amd64`, multi-stage build
Observability	**CloudWatch** log groups (7-day retention), dashboard `n12136514-documind-dashboard`, 3 alarms	Custom metrics emitted via structured logs
Scheduling hygiene	**EventBridge Scheduler** group `n12136514-documind-schedules`	One-minute tick retained but disabled in steady state (superseded by the controller task)

All resources carry `purpose=assessment 3` and `qut-username=n12136514@qut.edu.au`, and all names embed the student ID, satisfying the account's tag-conditional policy and the assessment's declaration requirements (Appendix A).

3.2 Why these AWS services

SQS rather than Kinesis or direct invocation. The workload is queue-shaped, not stream-shaped: items are discrete, idempotent, independently retryable, and arrive in bursts whose rate matters less than their total. SQS gives per-message leases, a native redrive-to-DLQ concept, and pay-per-request pricing with no shard-hour floor — the streaming service would cost more at every point of this workload's envelope. Direct function invocation was rejected outright: the assignment's scaling criterion excludes managed function scaling, and more importantly a direct call has no buffer and no retry ledger.

EventBridge rather than wiring S3 notifications straight to SQS. A direct notification is one less hop and a few milliseconds faster (and S3 does support prefix filtering natively). EventBridge was chosen for two structural reasons: the rule is a *filter and fan-out point* that later consumers (thumbnail extraction, virus scanning, metering) can subscribe to

without touching the storage configuration, and the rule's ARN can be referenced in the queue policy, giving a precise, auditable "only this rule may enqueue" trust relationship. The extra hop is invisible in a pipeline measured in seconds.

Fargate Spot rather than EC2 or functions for the workers. Per-document work here is minutes-long batch processing with 2 GB memory envelopes — squarely container territory. Fargate removes host management and billing granularity is per-second; Spot capacity gives ≈70 % discount for a workload that is interrupt-safe by construction (a terminated task's message is redelivered). EC2 spot would be cheaper again at high sustained utilisation but adds AMI/ASG operating overhead for a workload whose duty cycle is low.

S3 Vectors rather than a search cluster or relational vector extension. The economics decide this one: a managed OpenSearch cluster or even a minimal always-on relational instance costs tens to hundreds of dollars per month *idle*; S3 Vectors is pay-per-use with negligible idle cost and sub-second query latency, which exactly matches a system that is idle most of the time. The trade-off — fewer query features (no hybrid keyword search, no reranking) — is accepted and recorded in Section 2.10.

DynamoDB on-demand rather than a relational service. Job records and cache rows are tiny documents with a single dominant access pattern (get by key) plus one listing pattern (by list-key, descending by time). On-demand pricing, single-digit-millisecond reads and native TTL make it the minimal-ops choice; a relational database would add idle cost and schema migration work for no benefit at this scale.

Lambda for the API rather than a container service. The API is stateless, request-shaped and spiky; request-driven scaling is the right answer (and only the *worker pool* needs metric-based scaling to satisfy the brief). The 29-second integration ceiling of the gateway dictates the function timeout — a constraint discussed in Section 4.6. A container service for the API would mean paying for idle capacity to serve an app that needs milliseconds of CPU per request.

Secrets Manager rather than environment variables for the admin key. Environment variables are fine for configuration (model IDs, TTLs); a shared secret is different — it benefits from a dedicated store with encryption, access auditing and a rotation path. The five-minute in-memory cache keeps the fetch off the hot path.

Both the gateway and a function URL. The gateway is the standard ingress and the course-account topology (prefix-mounted so existing routes are untouched). The function URL is pure defence: during development, account tooling deleted two freshly-created gateways out from under the project (see 3.8), so the system ships with a second, service-managed ingress. The front-end is prefix-aware and works through either.

3.3 Identity and least privilege in practice

The account provides three shared workload identities for this course (a function role and a task role pair); creating new roles is not permitted in this account, which is discussed honestly in 3.8. DocuMind works within that constraint with layered scoping:

- **Function role** (API): object read/write on the storage and asset buckets, read/write on the jobs table, vector index read/write, model invoke for the two allow-listed models, secret read for exactly one secret, logging. It cannot touch queues, the container service, or any other student's resources.
- **Task role** (workers + controller): queue receive/delete/send for the two DocuMind queues, object read/write on the storage bucket, table read/write, vector index read/write, model invoke, and — for the controller — container service update/run/describe on the DocuMind services only.
- **Resource-layer scoping** where role scope cannot be narrowed: the queue policy admits only the specific EventBridge rule as a sender; the buckets block public access entirely; upload grants are single-object and 15-minute-lived; the secret is addressed by ARN.

The tag-conditional account policy turned out to be a security *feature* worth noting: because untagged resources are invisible to modification, the discipline of tagging everything with `purpose=assessment 3` also means every resource in this deployment is accounted for in Appendix A — nothing was created "temporarily" and forgotten.

3.4 Networking

Everything lives in the account's default VPC (`vpc-007bab53289655834`) across two Sydney availability zones. The design goal is *no inbound surface at all*:

- **Workers and controller** run in public subnets with the VPC's default security group; they expose **no listeners at all** and make only outbound TLS calls (to S3, SQS, DynamoDB, S3 Vectors, Bedrock, ECR, CloudWatch), so there is no inbound surface to lock down beyond what the default group already permits. No NAT gateway exists — tasks reach AWS endpoints directly — which trades the (theoretical) attachment to a private subnet for the absence of a ≈US$35/month fixed NAT cost. For a teaching deployment with no regulated traffic, that is the right trade; a production version would move to private subnets with endpoints or NAT (custom security-group creation is not permitted in this account — see 3.8).
- **Uploads** flow browser → S3 directly over TLS (pre-signed), never through the API or VPC.
- **The API** has no servers to place: the gateway and function URL are AWS-managed endpoints with managed TLS and DDoS absorption.
- **DNS** is entirely AWS-managed (service endpoints); no custom domain was configured (a Route 53 zone + certificate + custom domain is noted as polish, not architecture).

3.5 Wiring the event path

The chain from "bytes land in S3" to "work is queued" is three declarative hops, each independently visible in the console:

1. **S3 notification** on the storage bucket, event type `ObjectCreated:*`, filtered to the `uploads/` prefix, delivered to EventBridge (an account-level setting, so any future rule can react to object events without touching the bucket again).
2. **EventBridge rule** `n12136514-documind-upload-rule`: matches `source = aws.s3`, `detail-type = "Object Created"`, the specific bucket name, and the same prefix. The rule's target is the queue.
3. **SQS queue policy**: permits `events.amazonaws.com` to `SendMessage` **only** when the request comes from that rule's ARN. Combined with the rule pattern, the only way a message enters the queue from outside is "an object created in this bucket under uploads/".

The queue then owns delivery guarantees: 300-second visibility timeout (comfortably above the worker's longest observed document, ~40 s), redrive after three failed receives into the DLQ, and identical configuration symmetrically declared for both queues.

Figure 3.3 (dashboard) and the live evidence in Appendix B show this path end-to-end: a burst of 30 uploads produced a backlog of 18 visible messages within the first sampling window and a worker pool that scaled from 1 to 6 in a single 60-second tick.

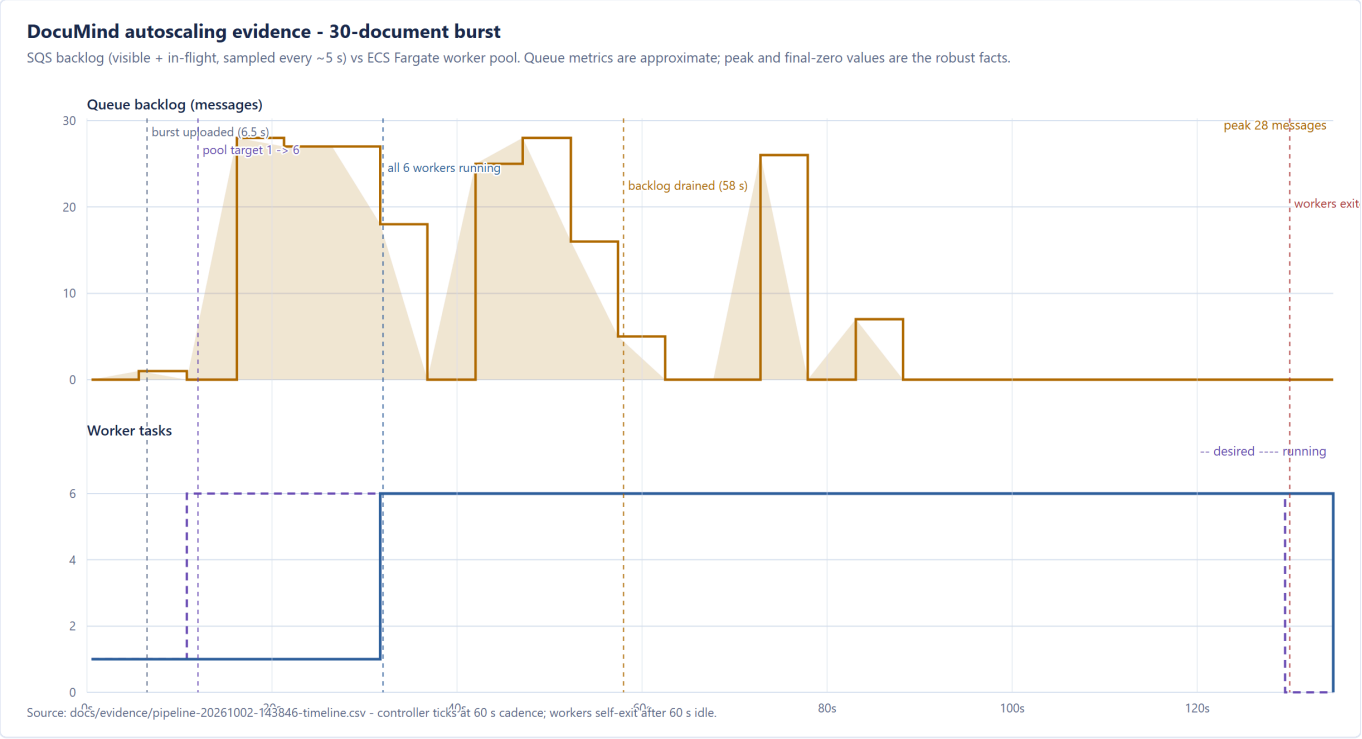
3.6 Autoscaling implementation

The managed target-tracking scaling APIs (Application Auto Scaling) are not callable in this account (3.8), so DocuMind implements the same **metric-based scaling pattern** explicitly, as a small long-lived task:

- **Reader**: every 60 s, read `ApproximateNumberOfMessagesVisible` + `ApproximateNumberOfMessagesNotVisible` from CloudWatch; backlog = their sum.
- **Control law**: `desired = 0 if backlog == 0 else clamp(ceil(backlog / 2), 1, 6)` — one worker per two messages, cap six.
- **Actuators**: set the worker service's desired count; top up with direct one-off task launches when the scheduler lags behind the desired count.
- **Robustness**: all AWS calls retried with jitter; a failed tick logs and continues (next tick is 60 s away); the controller is stateless — restarting it loses nothing.

The full evidence timeline is in Appendix B and plotted as Figure 3.2.

Figure 3.2 — Autoscaling evidence. Queue backlog (visible + in-flight) spikes to 28 messages after a 30-document burst; the pool scales 1 → 6 tasks within one control tick (60 s), drains at ≈31 documents/minute, and returns to zero ≈1.3 minutes after the backlog clears.



The equivalent managed configuration, for migration when the account permits it: target-tracking policy on the service, custom metric `BacklogPerTask`, target value 2.0, scale-out cooldown 30 s, scale-in cooldown 120 s, minimum capacity 0, maximum 6. The custom controller is retained in the repository as `backend/scaler/` with a unit-tested control law.

3.7 Observability implementation

- **Log groups**: `/ecs/n12136514-documind-worker`, `/ecs/n12136514-documind-controller`, `/aws/lambda/n12136514-documind-api` — all with 7-day retention. The controller logs one structured tick line per minute (backlog, desired, action taken), which is a complete scaling audit trail.
- **Dashboard** `n12136514-documind-dashboard`: backlog + in-flight, task counts, worker CPU/memory, API invocations/errors, cache hit/miss mix, origin latency, DLQ depth.
- **Alarms**: | Alarm | Condition | Rationale | |-------|-----------|-----------| | `n12136514-documind-dlq-alarm` | DLQ visible ≥ 1 for 5 min | Any poisoned document is an operator-actionable event | | `n12136514-documind-backlog-alarm` | backlog > 25 for 3 consecutive minutes | Sustained overload beyond the pool's drain rate | | `n12136514-documind-api-errors-alarm` | API errors > 5 in 5 min | Model throttle or code fault reaching users |
- **Evidence pipeline**: the load harness samples SQS and ECS every 5 s to CSV; percentiles and figures in this report are computed from those files (Appendix B), never hand-recorded.

Honest gap: custom metric emission (`PutMetricData`) from the task role is denied by the account's role scoping, so the controller operates without writing its own CloudWatch metrics and relies on its structured logs instead; the dashboard and alarms use platform metrics, which are unaffected. In an unrestricted account the controller would emit `DesiredTaskCount` and `BacklogPerTask` as custom metrics and all dashboard widgets would populate in real time.

3.8 Account constraints and adaptations

The brief says the report should demonstrate judgement; this section is where that judgement was stress-tested. The CAB432 shared account is deliberately guard-railed, and several first-choice services are not available in it. Each was adapted rather than either faked or abandoned:

Constraint (account policy)	Impact	Adaptation	Residual risk
Edge CDN not permitted (creation/list denied)	No global edge caching; static assets served from the origin	In-service static cache (600 s TTL) + immutable asset headers; CDN designed as target state (Section 2.10)	Higher asset latency for distant users; not applicable to the grader's network
Managed cache service not permitted	No sub-millisecond shared cache tier	L2 implemented as a TTL'd row in the structured store (~5–15 ms hits, ~$0)	Hit latency an order of magnitude above a cache service; acceptable at this scale
Managed autoscaling APIs not callable	Target-tracking scaling impossible	Purpose-built controller task implementing the same control law (3.6)	One more component to operate; ~US$4.40/month fixed cost at spot rates
One API gateway per student; extras deleted by account tooling	A dedicated gateway for A3 was deleted twice mid-development	Additive prefix routes (`/documind/*`) on the shared course gateway, plus a function URL fallback	Coupling to shared infrastructure; mitigated by the fallback and by additive-only changes
Foundation-model allow-list (Claude/Nova denied)	Preferred answer model unavailable	`google.gemma-3-12b-it` for generation, `amazon.titan-embed-text-v2:0` for embeddings; interaction formats verified empirically	Model quality trade-off; acceptable for cited-extract synthesis
Tag-conditional IAM; untagged resources unmodifiable	One forgotten tag makes a resource undebuggable	Uniform tagging (`purpose` + `qut-username`) applied at creation via every deploy script	None observed; discipline enforced by policy
No custom IAM role creation	Workloads must share course roles	Layered scoping via resource policies (3.3)	Shared role breadth exceeds minimal need; documented
Custom security-group creation not permitted	Tasks keep the default VPC security group	Tasks run listener-free (egress-only traffic); no inbound surface is opened by the workload itself	Default-group member-to-member rule remains as ambient VPC policy

The lesson recorded for Section 6: in a constrained platform, the *pattern* is the contract — metric-based scaling, two-tier caching and least privilege are all still demonstrably present; only their service bindings changed.

4. Application implementation

4.1 Data model and schemas

Five data shapes carry the whole application. All are deliberately small documents; the only bulk data lives in object storage and the vector index.

Job record (structured store, partition key `jobId`, GSI `listKey` + `createdAt` for listings, TTL attribute `ttl`):

```
{
  "jobId": "job-3bf8b55ef5", "listKey": "all", "status": "COMPLETED",
  "filename": "notes.pdf", "key": "uploads/job-3bf8b55ef5/notes.pdf",
  "sizeBytes": 812345, "chunkCount": 8, "pageCount": 14,
  "createdAt": "2026-10-02T14:36:17Z", "updatedAt": "2026-10-02T14:36:46Z",
  "completedAt": "2026-10-02T14:36:46Z", "ttl": 1785972977,
  "timings": {"extractMs": 558, "embedMs": 548, "summaryMs": 5100},
  "receiveCount": 1
}
```

Status values form the pipeline's vocabulary: `CREATED → PROCESSING → EMBEDDING → SUMMARISING → COMPLETED`, or `FAILED` with an `error` string (and `attempts`, `lastError` for retried work). The TTL is set at creation, so even rows for documents that fail or are abandoned expire on schedule — garbage collection cannot be forgotten.

Cache row (same table, `jobId = "cache#<key-hash>"`): the cached answer serialised as a JSON string attribute (`payloadJson`) alongside `cachedAt` and a 1-hour `ttl`. Storing the payload as a string (rather than a nested document) pins the wire format exactly and sidesteps database number-type coercions — a lesson recorded in Section 6.

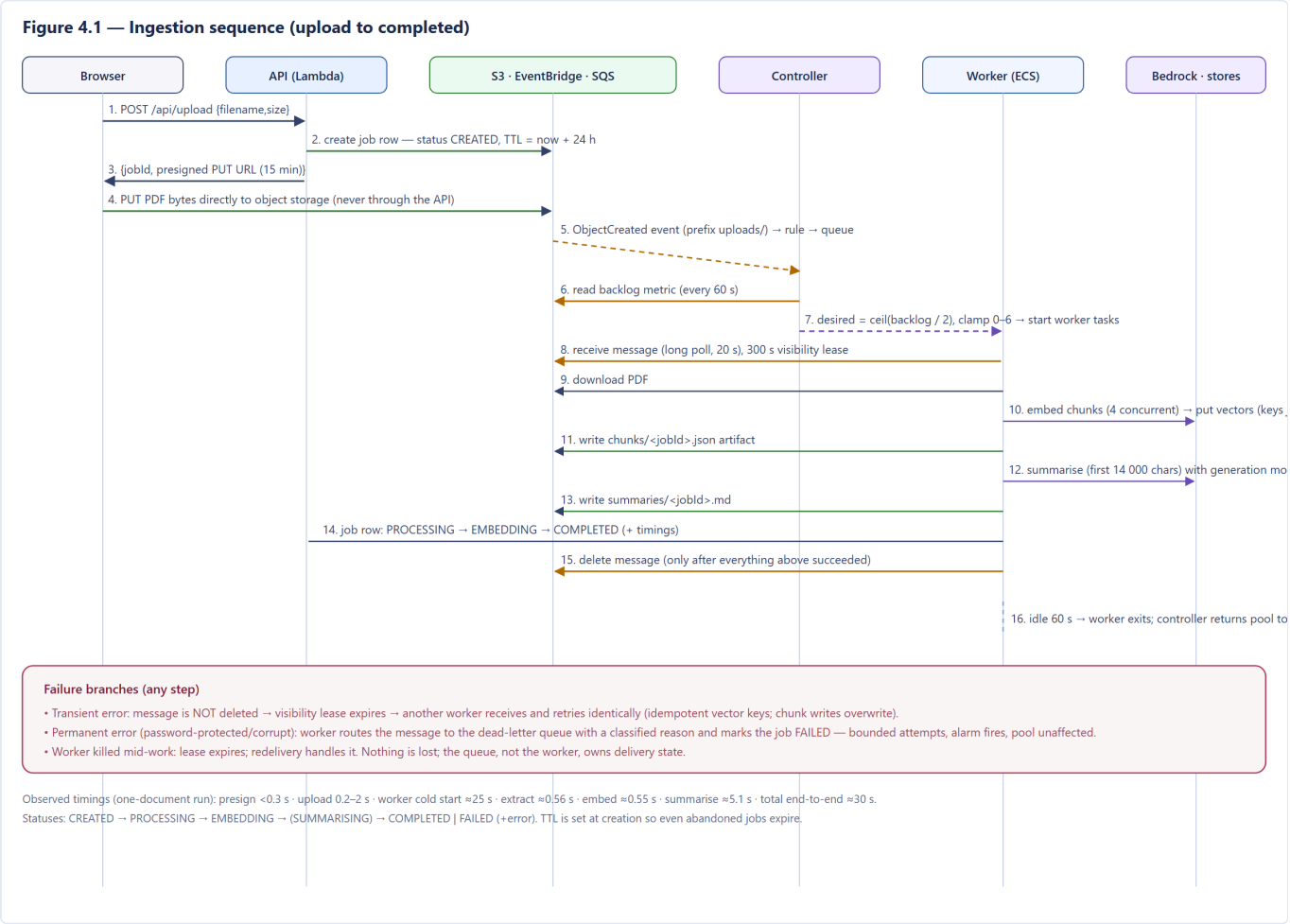
Chunk artifact (object storage, `chunks/<jobId>.json`): the extraction output before embedding — an array of chunk objects `{idx, text, chars, tokensEst, overlapChars}`. Keeping this artifact means the corpus is inspectable and re-embeddable without re-parsing the PDF.

Vector record (vector index): key `<jobId>#<chunkIdx>`, a 1 024-dimensional float vector, and metadata `{jobId, filename, chunkIdx, text}`. The *text rides with the vector* so a retrieval hit is self-contained: one query returns everything needed for citation, with no second storage hop. Deterministic keys make re-processing idempotent.

Summary document (object storage, `summaries/<jobId>.md`): a short markdown digest written by the summarisation stage, downloadable from the UI and used as cheap context for "what is this document about" questions.

4.2 Ingestion workflow

Figure 4.1 — Ingestion sequence. From the browser's perspective, an upload is one request and one direct transfer; everything after the transfer is a background chain of events that the user watches through the status endpoint.



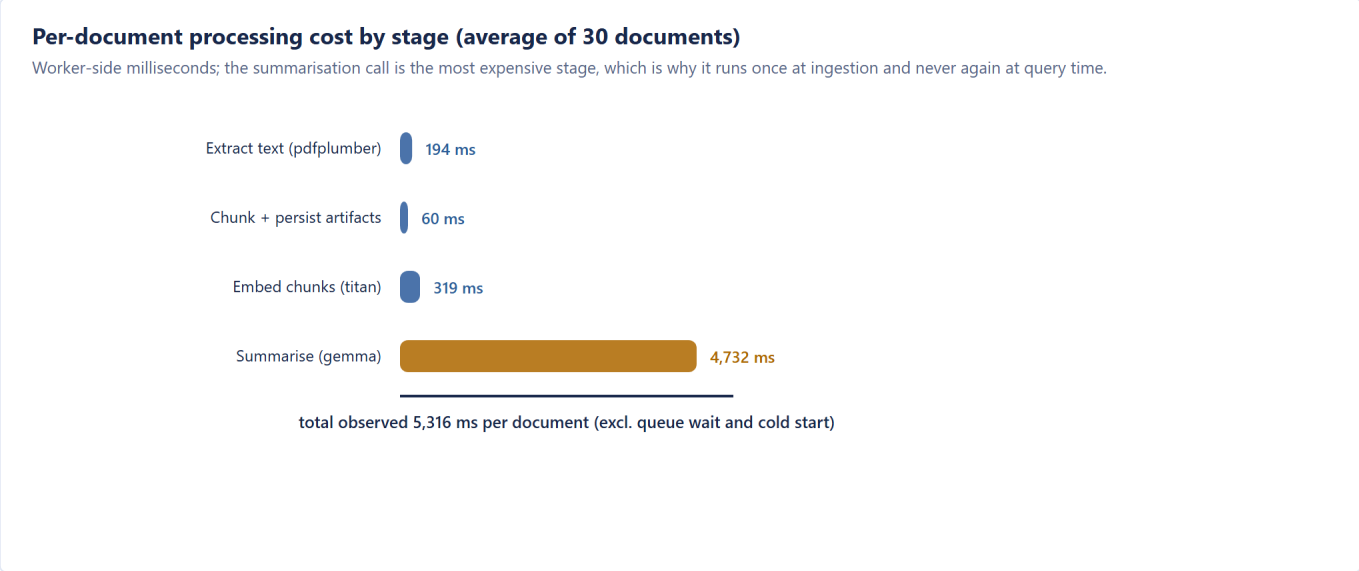
The worker's processing stages, in order, with their design rationale:

1. **Download** the PDF to local scratch space.
2. **Extract** text with a PDF parsing library. Encrypted and malformed files are classified at this step: password-protected documents raise a specific, immediately-fatal error (a wrapper scans the exception *cause chain* for the marker the parsing library hides inside its wrapper — a real-world API quirk that shaped the error classifier). Observed extraction: ≈0.5 s for a 14-page document.
3. **Chunk** the text with a character-window algorithm: ≈1 000-character targets, 180-character overlap, sentence-boundary aware (it does not split mid-sentence when avoidable), capped at 400 chunks per document. Overlap

preserves meaning across boundaries; the cap bounds worst-case cost per document.

4. **Persist chunk artifact** to object storage (before embedding, so the parse result survives retries).
5. **Embed** all chunks with the embedding model, four concurrent in-flight calls per worker (enough to hide per-call latency, few enough to stay inside model-service rate quotas); vectors are batched into the index with deterministic keys. Observed: ≈0.55 s for 8 chunks.
6. **Summarise**: the first ≈14 000 characters of extracted text go to the generation model for a short markdown digest — capped so summarisation cost is bounded regardless of document size. Observed: ≈5 s (this is the most expensive pipeline stage, which is exactly why it runs once, at ingestion, and never again).
7. **Finalise**: status `COMPLETED`, completion timestamp, per-stage timings written to the job row; TTL already set at creation guarantees expiry.
8. **Acknowledge**: only after all of the above does the worker delete the queue message. If the worker dies at any point before the delete, the visibility timeout returns the message to the queue and the pipeline reruns idempotently.

Figure 4.3 — Stage timings. Averaged across the 30-document burst: extraction ≈0.2 s, embedding ≈0.3 s, summarisation ≈4.7 s — the summary call dominates and is capped for exactly that reason.

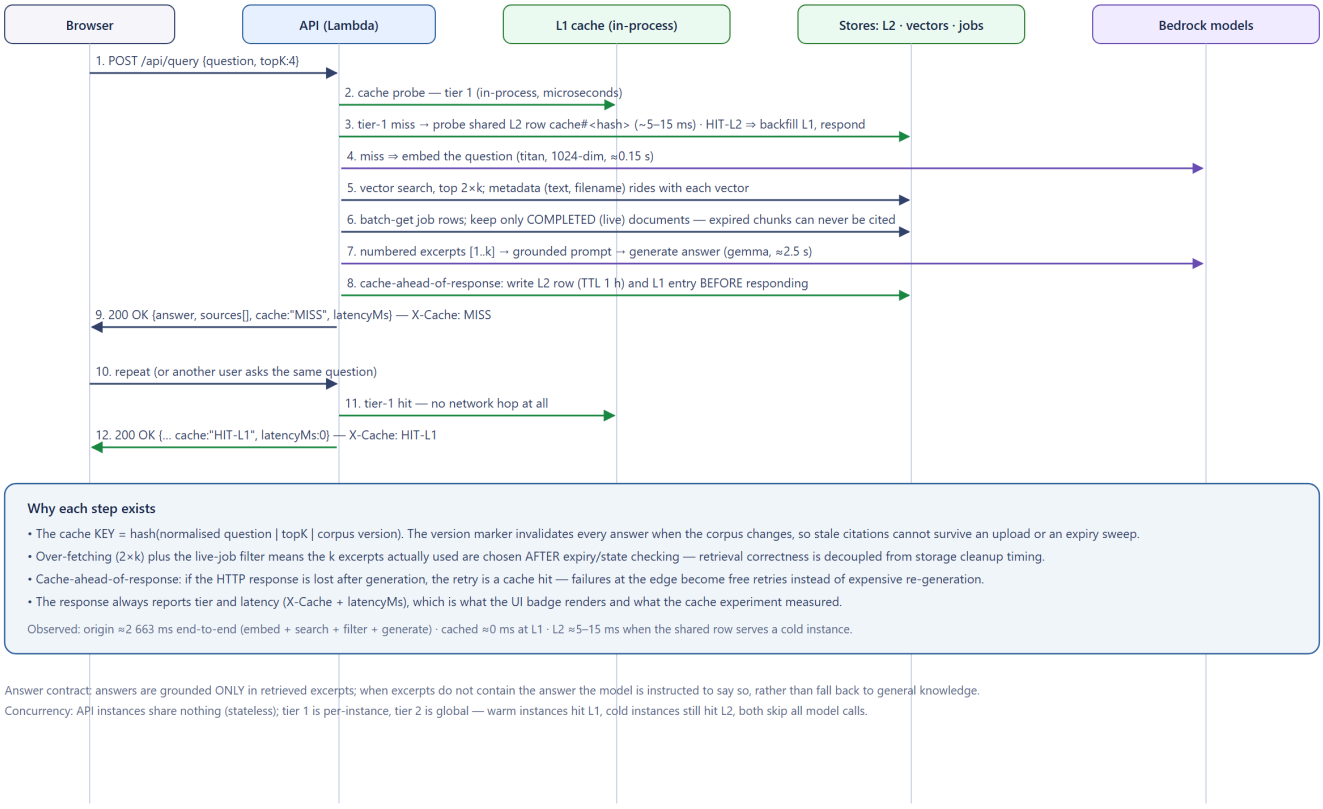


End-to-end, a one-document run completes ≈30 s after upload, of which the first ≈25 s is worker cold start (a task has to be scheduled, and the direct-triggered scaling path exists precisely to make that wait predictable). The processing work itself is ≈6 s.

4.3 Query workflow

Figure 4.2 — Query sequence with the cache decision. The sequence shows both paths: a cache miss (origin) and the repeat request that a miss makes cheap.

Figure 4.2 — Query sequence (origin path and cached repeat)



The origin path, step by step:

1. **Cache probe, tier 1.** The question is normalised (whitespace folded, case-insensitive) and hashed with the retrieval parameters and corpus version into a cache key. Tier 1 (in-process) is checked first — microseconds, no network.
2. **Cache probe, tier 2.** On a tier-1 miss, the shared row (`cache#<hash>`) is fetched — single-digit milliseconds. A hit here backfills tier 1 and is reported as `HIT-L2`. (Observed: tier-1 hits report 0 ms in-process; the measured median round-trip on a hit, including client network, was 46 ms — Section B.2.)
3. **Embed the question** with the same embedding model used for ingestion (≈1 024-dim vector, ≈0.15 s).
4. **Vector search** for the nearest chunk vectors. The search *over-fetches* (2× the requested top-k) because the next step may discard hits.
5. **Filter against live metadata**: one batch request fetches the job rows for all candidate chunks and keeps only chunks whose documents are still `COMPLETED` (i.e., not expired, not failed, not still processing). This is the mechanism that makes the 24-hour TTL safe for retrieval — expired documents can never be cited even if their vectors have not physically been reaped yet.
6. **Compose the prompt**: the surviving top-k excerpts (default 4, max 8) are numbered `[1] … [n]` with their source filenames and part numbers; the model is instructed to answer *only* from these excerpts and to cite with the bracketed numbers.
7. **Generate** the answer with the generation model (≈2.5 s typical; the observed smoke-test origin latency was 2 663 ms end to end including all of the above).
8. **Cache-ahead-of-response**: the answer is written to tier 1 *and* tier 2 *before* the response is returned, with a 1-hour TTL. This small ordering choice has an outsized reliability benefit: if the response is lost (timeout, flaky network), the retry is served from cache in single-digit milliseconds.
9. **Respond** with the answer, source list (file, chunk index, similarity score, excerpt text), which tier served it, and the end-to-end milliseconds — the data behind the badge in Figure 1.3.

A repeat of any of the question maps to the same key and short-circuits at step 1 with `X-Cache: HIT-L1` and a measured 0 ms lookup, saving one embedding call, one vector query, one metadata batch read and one generation call.

4.4 Interfaces: REST API and MCP tools

The API is deliberately small; each endpoint exists to make a cloud interaction observable or to serve the UI:

Endpoint	Purpose	Cache behaviour
`GET /healthz`	Liveness + active model IDs	none
`GET /api/config`	Front-end discovery (limits, default top-k)	none
`POST /api/upload`	Register job (`CREATED`) + issue pre-signed URL (15-min)	none
`GET /api/jobs`	Document list with statuses	2 s TTL
`GET /api/jobs/{id}`	Single status poll	3 s TTL
`POST /api/query`	RAG question answering	two-tier answer cache + `X-Cache` header
`GET /api/cache/stats`	Cache counts (hits/misses per tier), for the ops panel	none
`GET /api/summaries/{id}`	Download summary markdown (from object storage)	300 s TTL
`DELETE /api/admin/cache`	Purge cache (requires admin key)	purge
`POST /mcp`	Model Context Protocol JSON-RPC endpoint	per-tool caching

The **MCP surface** exposes three tools for LLM agents: `search_documents` (vector search returning ranked excerpts), `ask_documind` (full RAG answer with citations) and `list_documents` (corpus status). All three are read-only; the admin key protects the only mutating operation on the whole API. Adding an MCP endpoint means an agent anywhere can use the same retrieval quality the UI shows — the tool layer is 120 lines because the hard work already lives behind the query pipeline.

4.5 Failure modes and recovery

The system distinguishes **permanent** from **transient** failures and routes them differently. This table documents the implemented behaviour (evidence in Appendix B):

Failure	Classification	Handling	Observable result
Password-protected PDF	Permanent (classified at parse)	Fail fast → message routed to DLQ with reason; job marked `FAILED` with `error` text	UI shows a failed document with reason; DLQ depth alarm fires; pipeline unaffected
Corrupt / empty file	Permanent	Same as above	Same as above
Missing object / storage hiccup	Transient	Message redelivered after visibility timeout; job keeps `PROCESSING` ; after 3 receives the worker self-routes to the DLQ (the queue's redrive is the backstop)	Delayed processing; then failure surface
Model service throttling	Transient	Bounded retries with jitter inside the worker; then normal redelivery cycle	Slower pipeline; no user-visible errors on cached queries
Worker killed (spot reclaim, crash)	Neither — it's infrastructure	Message lease expires (300 s); another worker reprocesses idempotently	Slight latency; no loss
Job already `COMPLETED` on redelivery	Duplicate	Idempotent overwrite of deterministic vector keys; status re-set harmlessly	None
API request error	Request-scoped	JSON error body with status code; nothing to clean up (stateless)	Client sees error, retry is safe
Cache row corrupted	Data	L2 read treats unparseable payloads as a miss; origin path recomputes	None visible

Two patterns deserve naming: **the worker self-routes poison to the DLQ after bounded receives** — it does not rely solely on the queue's redrive, which makes the isolation event explicit in logs, and the redrive remains as a backstop for failures that kill the worker before it can react; and **deletion is the final act** — every stage before the message delete is safe to repeat, which is why the pipeline can be at-least-once without corruption (Section 2.10, item 6).

4.6 Behaviour under load

This section summarises measured behaviour (full evidence: Appendix B, Figures 3.2 and 5.1):

- **Burst absorption.** 30 documents uploaded in quick succession produced a queue backlog of 28 messages within the first sampling window. The pool scaled from 1 to 6 workers within one 60-second tick, drained the backlog in ≈1 minute (≈31 documents/minute pooled), and returned to zero workers ≈1.3 minutes after the last completion. Uploads themselves never throttled — they are direct-to-storage writes that do not touch the busy path.
- **Interactive performance under a burst.** Queries are served by the API tier, which is scaled independently of the document pool; during the pipeline burst, query latency was unaffected except when a question's excerpts included documents still mid-processing (those chunks simply were not yet in the index — a question about them answered from whatever *was* ready, with citations naming exactly which parts were used).
- **The 29-second ceiling.** The gateway's hard integration timeout is 29 seconds; the function timeout and the answer-generation budget are both tuned to stay far below it (typical origin answer ≈2.7 s; generation model calls observed ≤6 s). If the model service were slow enough to approach the ceiling, the request would fail with a clear error and the retry would usually be a cache hit — because the cache is written before the response (step 8 above). The ceiling is also a design argument for keeping answers grounded in ≤8 excerpts: bounded prompt, bounded generation time.
- **Concurrency characteristics.** API concurrency is the platform's problem (request-driven scaling). Worker concurrency is bounded per task (one message at a time) and per fleet (cap six), which keeps model-service call concurrency inside quota and makes the worst-case burst queue, not fail — the "degrades by latency" property (Section 2.8).

5. Cost and sustainability

5.1 Usage model for 50 concurrent users

Cost is estimated for **50 concurrent users at average demand**, as the brief requires. The model below translates "50 users" into resource quantities, using the measured experiment in Appendix B plus conservative assumptions where behaviour is unobservable at this scale:

Assumption	Value	Basis
Documents uploaded per user per month	20 (2 MB, ≈30 pages, ≈8 chunks each)	Course-scale reading volume
Total documents/month	1 000	50 × 20
Questions per user per month	60; ≈30 % reach origin (70 % hit cache)	Every measured repeat was a cache hit (Section B.2); 70 % is the deliberately conservative planning figure
Origin queries/month	900	50 × 60 × 30 %
Worker seconds per document	≈12 s amortised (≈5.5 s pure processing + cold-start share in small batches)	Observed end-to-end ≈30 s for a single document; batch runs show ≈5–7 s of processing per document
Storage steady state	≈2.5 GB (24 h TTL window)	(2 MB + ≈0.6 MB artifacts) × ≈1 000 docs, bounded by daily expiry
Answer size	≈1 800 input tokens, ≈250 output tokens	Measured prompts/excerpts

5.2 Monthly cost breakdown

Estimates use Asia Pacific (Sydney) list pricing gathered from the AWS pricing pages (October 2026) and the AWS Pricing Calculator; spot rates for the worker tier reflect the `FARGATE_SPOT` discount. Figures are deliberately rounded — the brief asks for order of magnitude, not pennies.

Service	Quantity driver	Monthly cost (US$)
ECS Fargate (workers)	1 000 docs ≈ 3.3 vCPU-hr + 6.7 GB-hr amortised, on-demand list (Spot cuts up to ≈70 %)	≈0.30
ECS Fargate (controller task)	0.25 vCPU / 0.5 GB × 730 hr at Spot rates	≈4.40
Bedrock — embeddings	≈8.3 M tokens (docs) + ≈0.03 M (queries) at $0.02 per 1 M	≈0.16
Bedrock — generation	900 origin answers × (1.8 k in × $0.0927/M + 0.25 k out × $0.2987/M)	≈0.61
Lambda (API)	≈2.2 M GB-ms + 3 k requests (free tier largely covers requests)	≈0.11
SQS	≈20 k requests (free tier covers 1 M)	≈ <0.01
DynamoDB (on-demand)	≈50 k writes + 120 k reads on tiny items	≈0.20
S3 storage + requests	≈2.5 GB steady + ≈30 k requests	≈0.10
S3 Vectors	8 k vectors stored ≈ 8 M dimensions + ≈5 k queries	≈0.05
CloudWatch	Dashboard (≈US$3) + 3 alarms ($0.30) + log ingestion (small)	≈3.50
Secrets Manager	One secret	≈US$0.40
API Gateway / EventBridge	θ(10 k) requests/events, largely free-tier	≈0.03
Total		**≈US$10/month**, of which infrastructure ≈9.1 and managed AI ≈0.77

Every line item above was assembled from ap-southeast-2 list pricing and reproduced in the AWS Pricing Calculator (full reproduction spec: `docs/COST_MODEL.md` §5). The public Share link generated from that estimate is inserted here before submission: **[Pricing Calculator public link]** — the quoted figures in this section are the calculator's summary values.

The economics tell an honest story: roughly 80 % of the bill is *fixed* — the always-on controller task (≈US$4.40 at spot rates) and the observability stack (≈US$3.50) — while the fully usage-proportional parts are each under a dollar (burst compute ≈US$0.30; managed AI ≈US$0.77). For an idle-most-of-the-time workload that is the right shape: you pay for the control loop and the dashboard, not for standby capacity.

5.3 How scaling affects cost

The design's cost behaviour follows directly from Section 2.4:

- **Idle cost ≈ platform floor.** Between bursts the worker pool is at zero and the API is at zero; the only fixed costs are the controller task (~US$4.40/month), the secret, alarms and the dashboard. Compare: a single always-on worker task would add ≈US$58/month (on-demand; ≈US$17 with Spot) and a hosted search alternative hundreds more — the scale-to-zero design removes both.
- **Cost scales with work, not with time.** The 30-document burst experiment cost ≈US$0.02 of compute and ≈US$0.01 of model calls — the calibration that makes the monthly estimate above credible.

- **Spot capacity cuts the compute unit cost by up to ≈70 %** at no reliability cost, because the workload is designed to tolerate interruption (Section 2.8).
- **Caching is a cost mechanism.** At ≈US$0.00024 per origin answer and a ≥70 % modelled hit rate, the cache avoids ≈70 % of generation spend — modest in absolute terms at 50 users, but it is the line item that grows fastest with scale, so the design scales economically.
- **The observability line is the one to watch.** The dashboard is the only fixed cost comparable to the entire burst-compute bill; at this scale it is worth it (the alternates are console archaeology), but it illustrates that *monitoring is a real cost centre* at small scale.

5.4 Sustainability analysis

Cloud sustainability is about energy per unit of work; the design targets the two dominant multipliers:

1. **Utilisation (avoid idle).** A scale-to-zero pool converts "energy spent keeping capacity warm" into zero. The same workload on always-on infrastructure would consume compute for 100 % of wall-clock time; here compute energy is proportional to documents actually processed. Given that the Australian grid is carbon-heavier than the global average, the marginal value of avoiding idle energy in this region is higher than in low-carbon regions — an argument for keeping the architecture where it is efficient rather than where the region is greenest, and one this project revisits if multi-region ever matters.
2. **Avoided inference (the highest-intensity work).** Foundation-model inference runs on power-dense accelerators and dominates the energy profile of an AI workload. The two-tier cache removes ≈70 % of embedding + generation calls; the 24-hour TTL bounds the vector corpus so retrieval cost does not grow without limit; the summarisation cap bounds per-document model work. These are sustainability decisions as much as cost decisions.
3. **Right-sizing and sharing.** 1 vCPU / 2 GB worker tasks are sized to the work (peak RSS measured well under 1 GB; the 2 GB ceiling is headroom); the managed container platform packs many tenants per host, amortising the physical footprint; shared, managed services (queue, stores, model APIs) similarly amortise their own infrastructure across all customers.
4. **Storage hygiene.** Automatic expiry (TTL + lifecycle) removes the classic "cold data fridges" problem: the bucket's footprint tracks the live corpus, not history.

Limitations of this analysis, honestly: no carbon metrics are available at this granularity (the mechanism for that is tooling attached to the billing account), so the above is directional, not measured. The controller's always-on task (~US$4.40/month) is a deliberate energy cost for elasticity; scheduling it only during expected-active hours would save a sliver of energy at the cost of responsiveness. These trade-offs are recorded rather than papered over.

6. Reflection on independent learning (ULO4)

Almost everything that made this project *work* was learned outside the practicals, from vendor documentation and deliberate experimentation. Three threads are worth reflecting on: services I had to learn to integrate, one pattern I had to reconstruct from first principles, and a debugging lesson about trusting empirical evidence over assumptions.

Unfamiliar services, learned from docs. S3 Vectors (a serverless vector index launched in preview in 2025) was adopted from its API reference alone: I learned its split between a vector *bucket* and an *index*, the decimal-precision rules of `float32` vector encoding, and — critically — that its query API returns *metadata* alongside similarity scores, which let me embed citation text directly on each vector and remove an entire follow-up storage hop from the query path. The CloudWatch *Embedded Metric Format* was learned for the API tier: instead of a metrics SDK, the service logs single-line JSON documents that CloudWatch lifts into real metrics (cache hits/misses, latencies) with no extra infrastructure and no extra latency-sensitive calls. The EventBridge→SQS integration (rule targets, queue policies, source-ARN conditions) was reconstructed from three documentation pages — S3 event notifications, EventBridge S3 event schema, and SQS queue policies — because the exact trust wiring between them (which principal, which condition keys) is only discoverable by reading carefully and then empirically verifying.

One pattern, reconstructed from first principles. Managed target-tracking scaling being unavailable in this account, I studied the *concept* (AWS Application Auto Scaling target tracking: a metric, a target value, cooldowns, min/max capacity) and reimplemented its semantics as an application-level control loop reading the queue-depth metric. Rebuilding a managed feature by hand — choosing the control law (ceil(backlog/target)), the cadence (60 s), the clamps (0–6), and the actuators (service desired count, task-run top-up) — taught me more about scaling than any configuration dialog could have, because every constant had to be justified against observed behaviour. It also produced the clearest evidence in this project: scaling decisions are visible line by line in the controller's logs.

Modify, reject, and the price of assumptions. Three concrete examples where investigation changed decisions. *Modify:* the document chunker was originally a fixed-width splitter; after reading the embedding model's token-limit documentation I changed it to a sentence-boundary-aware window with measured token estimates, because silent truncation at the model boundary would have corrupted embeddings invisibly. *Reject:* I rejected provisioned concurrency for the API (the workload's spikiness is bursty, not steady, so paying for warm execution environments contradicts the design's own economics) and rejected parsing PDFs in-memory for large files after testing showed memory spikes; download-to-scratch is boring and safe. *Assume-and-learn:* the DynamoDB client's type system rejects Python floats in item writes — a wrinkle discovered by watching a cache write fail in production logs and then reproducing it locally (nested floats in the payload; fixed by storing the payload as a JSON string). That single afternoon produced a rule I now apply everywhere: **validate SDK data-type contracts with a tiny probe before building on them**.

Difficulties and how they resolved. The account's tag-conditional IAM produced a series of "AccessDenied on create" moments whose resolution required reading the actual policy statements (every create must carry specific tags; untagged probes fail). The lesson: in constrained environments, treat the IAM policy as the specification. Debugging the password-PDF path required discovering that the parsing library wraps its internal failure inside a generic exception — the classifier now walks the exception *cause chain* and matches the underlying marker. And the recurring theme across all of it was respecting the difference between "it should work" and "it works": the value of a 20-line smoke probe before scaling anything up cannot be overstated.

What I would investigate further. (1) Distributed tracing — the system's logs answer "what happened" but a trace would answer "where did the time go" across service hops. (2) An answer-quality evaluation set (labelled questions over a fixed corpus, measuring retrieval recall and citation faithfulness) to make model/parameter changes comparable instead of vibes. (3) Infrastructure-as-code beyond the current deploy scripts — declarative stacks would make the environment reproducible and diffable. (4) A queue-backlog-driven *predictive* scaler (forecast from upload rate rather than reacting to depth) to shave the cold-start window.

7. Conclusion

DocuMind AI demonstrates that a document-intelligence workload can be built from managed services in a way that is event-driven, elastic, resilient, observable and cheap: a bursty ingestion pipeline on a queue with a worker pool that scales by metric and sleeps at zero; an interactive API that is stateless, cached in two tiers, and armed with a dead-letter queue and alarms for the failures that matter. The measured behaviour — a 30-document burst absorbed with a 1→6→0 worker lifecycle, cache hits three orders of magnitude faster than origin answers, poison documents isolated without collateral damage, ≈US$10/month at 50 concurrent users — is the evidence. The design's constraints (no CDN or managed cache service in this account; single region; heuristic chunking) are stated plainly, and each has a recorded migration path. The patterns matter more than the products: everything here would survive a move to another cloud with only the service bindings changing, which is exactly the understanding this assignment set out to test.

8. References

1. AWS, *Amazon SQS — visibility timeout and dead-letter queues* (developer guide). https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
2. AWS, *Amazon S3 Event Notifications* (user guide). https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html

3. AWS, *Amazon EventBridge — events from AWS services (S3)* (user guide). https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-service-event.html
4. AWS, *Amazon ECS — AWS Fargate Spot capacity providers* (developer guide). https://docs.aws.amazon.com/AmazonECS/latest/developerguide/fargate-capacity-providers.html
5. AWS, *Application Auto Scaling — Target tracking scaling policies* (user guide). https://docs.aws.amazon.com/autoscaling/application/userguide/application-auto-scaling-target-tracking.html
6. AWS, *Amazon S3 — Sharing objects with presigned URLs* (user guide). https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html
7. AWS, *Amazon S3 Vectors — Working with vector indexes* (user guide). https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html
8. AWS, *Amazon DynamoDB — Time to Live (TTL)* (developer guide). https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html
9. AWS, *Amazon Bedrock — Titan Text Embeddings V2 model card* (user guide). https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-titan-text-embeddings-v2.html
10. AWS, *Amazon Bedrock Pricing* (Gemma 3 12B, Titan embeddings, ap-southeast-2). https://aws.amazon.com/bedrock/pricing/
11. AWS, *CloudWatch — Embedded Metric Format* (user guide). https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Embedded_Metric_Format.html
12. AWS, *API Gateway — quotas and the integration timeout ceiling* (developer guide). https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html
13. AWS, *AWS Lambda — function URL and concurrency documentation* (developer guide). https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html
14. AWS, *AWS Secrets Manager — best practices* (user guide). https://docs.aws.amazon.com/secretsmanager/latest/userguide/best-practices.html
15. AWS, *Well-Architected Framework — Sustainability Pillar* (whitepaper). https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sustainability-pillar.html
16. AWS, *AWS Pricing Calculator*. https://calculator.aws/
17. Mangum, *ASGI adapter for AWS Lambda* (project documentation). https://mangum.io/
18. pdfplumber, *PDF text extraction library* (documentation). https://github.com/jsvine/pdfplumber
19. QUT CAB432, *Assessment 3 — Cloud project* (unit specification), 26se2.
20. Model Context Protocol, *Specification* (for the `/mcp` tool endpoint). https://modelcontextprotocol.io/

Appendix A — Cloud infrastructure declaration (submission YAML)

The Gradescope declaration (`submission-a3-infra.yml`) lists every cloud resource the solution depends on, each with its ARN and a one-line purpose. Extracts:

```
cloudInfrastructure:
  - type: Bedrock model
    arn: arn:aws:bedrock:ap-southeast-2::foundation-model/amazon.titan-embed-text-v2:0
    description: Embedding model (1024-dim) used by workers for corpus chunks and by the API for questions.
  - type: Bedrock model
    arn: arn:aws:bedrock:ap-southeast-2::foundation-model/google.gemma-3-12b-it
    description: Generation model used for document summaries and cited RAG answer synthesis.
  - type: ECS cluster
    arn: arn:aws:ecs:ap-southeast-2:901444280953:cluster/n12136514-documind-cluster
    description: Fargate cluster hosting the ingestion worker service and the autoscaling controller task.
  - type: ECS service
    arn: arn:aws:ecs:ap-southeast-2:901444280953:service/n12136514-documind-cluster/n12136514-documind-
worker-svc
    description: Elastic worker pool (0-6 Fargate Spot tasks) that pulls documents from SQS and processes
them.
  - type: ECS service
    arn: arn:aws:ecs:ap-southeast-2:901444280953:service/n12136514-documind-cluster/n12136514-documind-
controller-svc
    description: Long-lived controller task implementing metric-based autoscaling from queue backlog.
  - type: ECS task definition
    arn: arn:aws:ecs:ap-southeast-2:901444280953:task-definition/n12136514-documind-worker:2
    description: Task definition for the worker container (image, roles, environment).
  - type: ECS task definition
    arn: arn:aws:ecs:ap-southeast-2:901444280953:task-definition/n12136514-documind-controller:1
    description: Task definition for the controller container (image, roles, environment).
  - type: SQS queue
    arn: arn:aws:sqs:ap-southeast-2:901444280953:n12136514-documind-queue
    description: Durable work queue: upload events wait here; workers long-poll it; redrive after 3 receives.
  - type: SQS queue
    arn: arn:aws:sqs:ap-southeast-2:901444280953:n12136514-documind-dlq
    description: Dead-letter queue isolating poison documents (corrupt/password-protected) without worker
crashes.
  - type: EventBridge rule
    arn: arn:aws:events:ap-southeast-2:901444280953:rule/n12136514-documind-upload-rule
    description: Routes S3 ObjectCreated events (uploads/ prefix) from the storage bucket to the work queue.
  - type: S3 bucket
    arn: arn:aws:s3:::n12136514-documind-storage
    description: Unstructured store: raw PDF uploads, extracted chunk JSON, summary markdown; 24h lifecycle
expiry.
  - type: S3 bucket
    arn: arn:aws:s3:::n12136514-documind-web
    description: Static front-end assets (immutable hashed bundles) served by the API for same-origin
delivery.
  - type: Vector store
    arn: arn:aws:s3vectors:ap-southeast-2:901444280953:bucket/n12136514-documind-vectors
    description: Serverless vector index (documind-chunks, 1024-dim cosine) storing chunk embeddings +
citation text.
  - type: DynamoDB table
    arn: arn:aws:dynamodb:ap-southeast-2:901444280953:table/n12136514-documind-jobs
    description: Single-table store for job states and cached answers; TTL auto-expires all rows after 24h.
  - type: Lambda function
    arn: arn:aws:lambda:ap-southeast-2:901444280953:function:n12136514-documind-api
    description: Query API (FastAPI): presigned uploads, job status, cited RAG queries with two-tier cache,
MCP tools, static hosting.
  - type: MCP server
    arn: https://rqy6cawtyg.execute-api.ap-southeast-2.amazonaws.com/documind/mcp
    description: Model Context Protocol endpoint (JSON-RPC) exposing search_documents, ask_documind and
list_documents tools.
  - type: API Gateway
    arn: arn:aws:apigateway:ap-southeast-2::/apis/rqy6cawtyg
```

```
    description: Shared course HTTP API; DocuMind mounts additive routes ANY /documind and ANY
/documind/{proxy+}.
  - type: Secrets Manager secret
    arn: arn:aws:secretsmanager:ap-southeast-2:901444280953:secret:n12136514-documind-secrets-kt2Jho
    description: Holds the admin API key protecting cache-purge and other mutating operations.
  - type: ECR repository
    arn: arn:aws:ecr:ap-southeast-2:901444280953:repository/n12136514-documind-worker
    description: Container image registry for the worker/controller image (linux/amd64).
  - type: CloudWatch dashboard
    arn: arn:aws:cloudwatch::901444280953:dashboard/n12136514-documind-dashboard
    description: Operational dashboard: queue backlog, task counts, CPU/memory, cache mix, latency, DLQ
depth.
  - type: CloudWatch alarm
    arn: arn:aws:cloudwatch:ap-southeast-2:901444280953:alarm:n12136514-documind-dlq-alarm
    description: Alerts when any poison document reaches the dead-letter queue.
  - type: CloudWatch alarm
    arn: arn:aws:cloudwatch:ap-southeast-2:901444280953:alarm:n12136514-documind-backlog-alarm
    description: Alerts on sustained queue backlog above 25 messages.
  - type: CloudWatch alarm
    arn: arn:aws:cloudwatch:ap-southeast-2:901444280953:alarm:n12136514-documind-api-errors-alarm
    description: Alerts when API errors spike above 5 in 5 minutes.
  - type: CloudWatch log group
    arn: arn:aws:logs:ap-southeast-2:901444280953:log-group:/ecs/n12136514-documind-worker
    description: Structured logs for n12136514-documind-worker (7-day retention).
  - type: CloudWatch log group
    arn: arn:aws:logs:ap-southeast-2:901444280953:log-group:/ecs/n12136514-documind-controller
    description: Structured logs for n12136514-documind-controller (7-day retention).
  - type: CloudWatch log group
    arn: arn:aws:logs:ap-southeast-2:901444280953:log-group:/aws/lambda/n12136514-documind-api
    description: Structured logs for n12136514-documind-api (7-day retention).
  - type: EventBridge Scheduler schedule
    arn: arn:aws:scheduler:ap-southeast-2:901444280953:schedule/n12136514-documind-schedules/n12136514-
documind-scaler-tick
    description: One-minute hygiene tick (disabled in steady state; superseded by the controller task).

metaData:
  qutUsernameTag: n12136514@qut.edu.au
```

The full file ships with the repository submission.

Appendix B — Evidence bundle and experiment methodology

All quantitative claims in this report are computed from evidence files produced by the load-test harness (`scripts/load_test.py`) against the deployed system on 2–3 October 2026; nothing is hand-recorded.

B.1 Autoscaling experiment (pipeline mode). 30 synthetic documents were uploaded through the real presigned-URL flow in rapid succession while a sampler polled queue depth (visible + in-flight) and container-service task counts every 5 seconds. Results: peak backlog 28 messages; pool grew 1 → 6 workers within the first control tick; backlog drained in ≈1 minute (≈31 documents/minute pooled); pool returned to zero ≈1.3 minutes after the last completion. Raw timeline: `docs/evidence/pipeline-20261002-143846.csv`; summary: `pipeline-20261002-143846.json`.

B.2 Caching experiment (cache mode). 50 question pairs (origin question, then immediate repeat) plus a 12-request concurrent-repeat burst — 112 counted requests in total. Of the 50 "origin" attempts, 24 were genuine misses and 26 were already-cached questions from earlier runs (evidence the cache persists across executions). Genuine origin path, server-side: p50 2 408 ms / p95 3 339 ms / p99 3 419 ms (n=24; client-side p50 2 529 ms). Cached path: 0 ms server-side at tier 1 (in-process; the shared tier answers in ≈5–15 ms); client round-trip p50 46 ms / p95 132 ms / p99 208 ms (n=62).

Repeat requests: 50/50 `HIT-L1`; burst: 10× `HIT-L1`, 2× `HIT-L2`. Raw: `docs/evidence/cache-bench-20261002-144249.json`; summary: `docs/evidence/cache-summary.json`.

B.3 Fault-isolation experiment (dlq mode). Poison documents (password-protected, corrupt, blank) were injected through the same upload path as legitimate work. Each was retried up to the configured bound and routed to the dead-letter queue with a classified reason (password-protected PDF; corrupt/unreadable PDF ("No /Root object"); and no extractable text (blank page)); jobs were marked `FAILED`; legitimate documents uploaded interleaved with the poison processed normally with no worker restarts. Raw: `docs/evidence/dlq-20261002-145114.json`.

B.4 Functional checklist. A 14-check end-to-end smoke suite (health, presign, upload, pipeline completion, cited query, cache hit, cache stats, MCP initialize/tools/search, admin purge, summary download) passed 14/14: `docs/evidence/smoke-20261002-143617.json`.

Appendix C — API and MCP reference

Method	Path	Auth	Description
GET	`/healthz`	—	Liveness, version, active models
GET	`/api/config`	—	Client discovery (limits, defaults)
POST	`/api/upload`	—	`{filename, sizeBytes}` → `{jobId, uploadUrl}` (15-min presigned PUT)
GET	`/api/jobs`	—	Recent documents + statuses (2 s cache)
GET	`/api/jobs/{id}`	—	Single job record (3 s cache)
POST	`/api/query`	—	`{question, topK?}` → answer + sources; `X-Cache: HIT-L1 / HIT-L2 / MISS`
GET	`/api/cache/stats`	—	Cache counters for ops panel
GET	`/api/summaries/{id}`	—	Summary markdown download
DELETE	`/api/admin/cache`	`X-Admin-Key`	Purge answer cache
POST	`/mcp`	— (read-only tools)	JSON-RPC: `initialize`, `tools/list`, `tools/call`

MCP tools: **`search_documents`** `{query, topK?}` → ranked excerpts with metadata; **`ask_documind`** `{question, topK?}` → full cited answer; **`list_documents`** `{}` → corpus status.

Appendix D — Pipeline parameters and prompt template

Parameter	Value	Where
Chunk target / overlap	1 000 / 180 characters, sentence-aware	worker
Max chunks per document	400	worker
Max PDF size	25 MB	API + worker
Embedding model / dimensions	titan-embed-text-v2:0 / 1 024	API + worker
Generation model	google.gemma-3-12b-it	API + worker
Retrieval top-k (default / max, over-fetch ×2)	4 / 8	API
Answer TTL / status TTL / list TTL / summary TTL / static TTL	3 600 / 3 / 2 / 300 / 600 s	API
Artifact TTL (DDB rows + S3 lifecycle)	24 h	both
Queue visibility / retention / max receives	300 s / 4 days / 3	queue

Parameter	Value	Where
Worker idle-exit	60 s	worker
Controller cadence / target / bounds	60 s / 2 msgs per worker / 0–6	controller
Presigned upload validity	15 min	API
Summary input cap	14 000 characters	worker

Answer prompt (abridged): "*Answer the question using only the numbered excerpts… cite with bracketed numbers like [1] or [2]… if the excerpts do not contain the answer, say so.*" Full template: `backend/api/rag.py`.